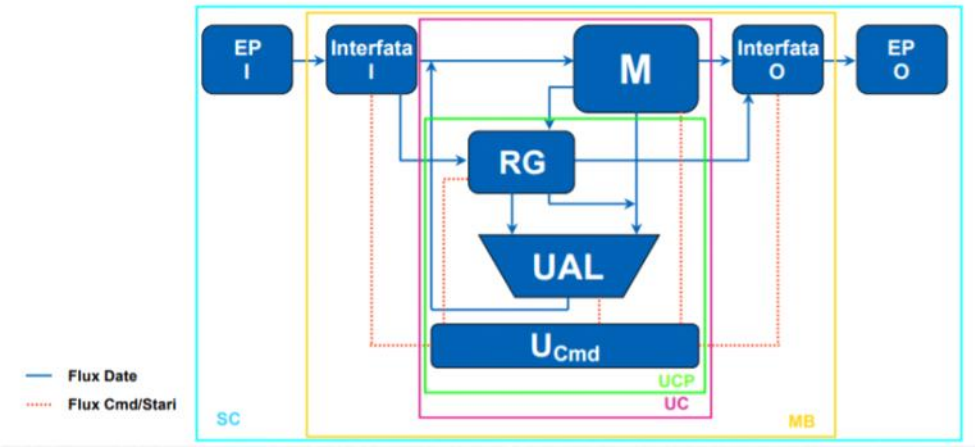
PMS si SBC

Friday, June 4, 2021 5:58 PM

PMS - Processor Memory Switches
SBC - Single Board Computers

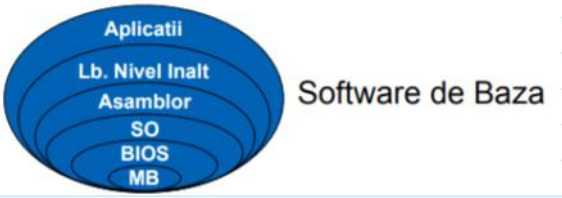
Arhitectura Von Neumann

→ are in centru unitatea aritmetico-logica (UAL) care creeaza o semantica noua
→ unitatea de comanda comunica cu restul componentelor prin flux de comanda/stari



- UCP = {RG, UAL, UCmd} = Unitatea Centrala de Prelucrare
- UC = {UCP, M} = Unitatea Centrala
- MB = {UC, I/E} = Masina de Baza
- SC = {MB, EP, Software de Baza} = Sistem de Calcul

→ Peste MB avem nevoie de specificatii despre cum arata - BIOS
→ BIOS-ul se ocupa de toate specificatiile care se intampla tot timpul la bootare
→ SO-ul abstractizeaza BIOS-ul ca sa ofere aceleasi servicii generice



Directive PMS

→ Processor Memory Switches
→ un mod de a descrie structural oricui fel de sistem numeric
→ fiecare primitiva PMS este caracterizata printr-un atribut si fiecare atribut are o valoare asociata SN (a1:v1, a2:v2 …. an:vn)

Elementele primare ale unui SN:

→ PC - procesorul central
- Citeste, interpreteaza si executa instructiuni masina
- In acest proces are loc generarea comenzilor spre toate resursele sistemului + citirea starilor acestora
- Atribute: functie, implementarea, formatul instr., lungimea instr., ciclu instr., tehnologie
- S-a facut saltul de la 32biti la 64 din cauza limitarii memoriei la 4GB

→ M - memoria
- Pastreaza informatia
- Actualizeaza informatia prin operatii de citire/scriere
- Necesita un sistem de adresare cu o corespondenta liniara intre adresa si continut
- Atribute: functie, tehnologie, operatii, mod acces, lungime cuvant, capacitate
- BIOS-ul este o memorie ReadOnly in general, e si Write cand intri in el
- Memoriile ROM sunt mai rapide decat RAM pt ca nu se mai face refresh pt a asigura actualizarea datelor

→ S - comutatorul/switch
- Asigura conexiuni intre componentele sistemului
- Poate evolua
  - de la structuri foarte simple = buffer 3 state de acces pe magistrala
  - la structuri complexe cu unitati de comanda proprii = comutatoare cu discipline de servire a cererilor de stabilire a legaturilor
- Atribute: structura, tip, numar de legaturi realizate, concurenta
- Motherboard-ul este un mare switch, o zona de tranzistori de pe procesor sunt switch-uri
- n procesoare conectate la m memorii conectate printr-un switch(maxim de paralelism: min(n, m))

→ L - linia de legatura
- Asigura legatura fizica intre diverse componente ale sistemului
- Nu prelucreaza informatia ci doar asigura traseul transferului spatial al datelor
- E fie o magistrala, fie o interfata seriala/paralela
- Atribute: functia, lungime cuvantului, mod de dialog, mod de control al accesului, standard

→ K - unitatea de comanda
- Componenta care exprima controlul in diverse subansamble ale sistemului
- Are o functie de comanda a resurselor asociate unitatilor functionale
- Nu citeste & interpreteaza instructiuni aritmetice logice, ci doar comanda & control
- Atribute: functia, impleentare, nr de stari(bistabil sau automat complex)
- Citireste, interpreteaza si executa instructiuni masina de comanda si control
- Setul de instructiuni: intructiuni universale, ciclu variabil, RISC cu ciclu fix, complex CISC

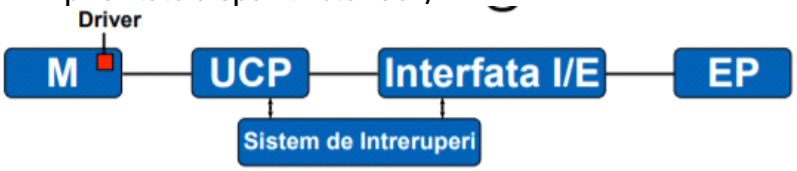
→ PI/E - procesorul de intrare/iesire sau interfata
- Poate fi incorporat in PC-ul in care are functie de I/E sau poate fi privit independent
- Are sens sa fie considerat independent pt ca nu vrem ca procesorul sa ruleze la viteza perifericelor
- Daca este o primitiva independenta are urmatoarele atribute: functie(DMA, canal I/E, procesor specializat I/E), tip implementare, rata de transfer(b/s), mod de transfer(rafala sau furt de ciclu) , lungimea cuvantului

Subsistemul de I/E
- asigura transferul datelor intre UC si mediul extern
- interfetele I/E asigura
  - adaptare electrica intre EP si UCP
  - sincronizarea temporala intre EP si UCP printr-un cuvant de stare (diferenta este de cateva ordine de marime)
  - transfer de date intre EP si UC:
    1. Transfer programat
    2. Acces direct la memorie (DMA)

3. Canal de I/E
4. Calculator de intrare/iesire (Front End Computer/Processor - FEP)

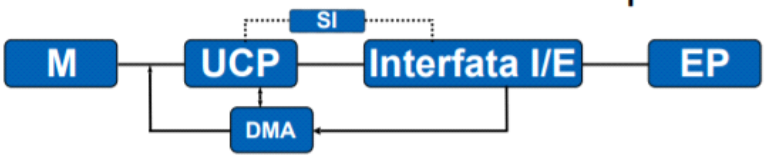
1. Transferul programat

→ UCP-ul se ocupa de fiecare caracter in parte
→ Driver-ul se ocupa de transferul datelor I/O (spune cum se interfateaza perifericul de sistemul de calcul)
→ Sincronizarea de face prin:
- citirea ciclica a starii(busy waiting): UCP asteapta dupa EP si driver-ul citeste si intrerupe cuvintele de date/stare/comanda(UCP merge la viteza EP)
- prin intreruperi: se dau ordine si se continua prelucrarile
  - rafala: interfata de I/E genereaza constant cereri catre SI
  - furt de ciclu: se pune o singura cerere catre UCP care da o mai mare prioritate dispozitivelor de I/E



2. DMA (Acces Direct la Memorie)

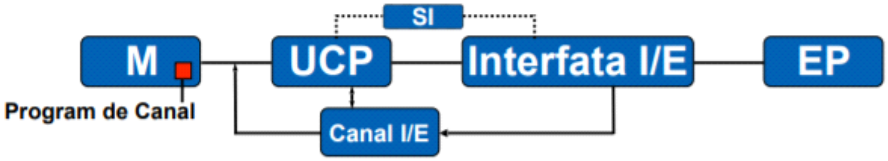
→ Asigura transferul de date intre EP si M fara interventia UCP



→ Dialogul intre UCP si DMA are loc doar la initiere & incheiere (prin transfer programat - intreruperi)
→ La initiere UCP-ul da DMA-ului adrese de date, sensul(R/W) si adresa EP
→ La orice eroare DMA cheama UCP-ul
→ UCP-ul nu poate initia un alt transfer pana nu s-a incheiat cel curent
→ Arbitrarea conflictului de acces la M al UCP/DMA se face prin furt de ciclu sau rafala

3. Canal de I/E

→ E un procesor specializat ce asigura transferul intre EP si M fara interventia UCP



→ Dialogul UCP – EP se face printr-un program de canal (din M – se da doar adresa de inceput a programului)
→ UCP poate inlantui mai multe programe de canal(mai bine ca la DMA care are un transfer odata)
→ Canalul aduce informatia necesara transferului in registrele proprii prin DMA (acces direct)
→ Canalul reincearca transferul in caz de eroare (nu mai trebuie handling ca la DMA)

4. Calculator de I/E (FEP)

→ Separa utilizatorii de SC, fara sa mai fie nevoie de interventia utilizatorului
→ Mecanism util pentru a nu irosi timpul unui procesor / calculator puternic
→ Ofera datele rapid si eficient pentru prelucrare
→ Este folosit des in sistemele multiprocesor
→ FEP-ul este o masina virtuala, nu una fizica

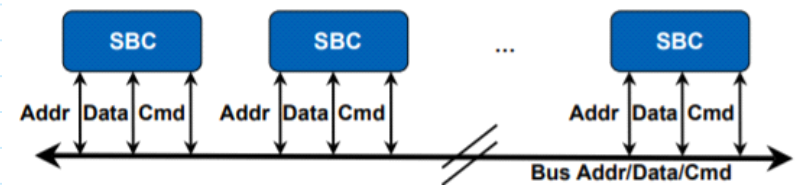
→ D / UAL - operatorul de date
- "Produce" unitati de informatie cu semnificatie noua
- Efectueaza operatii aritmetice & logice + prelucrari primare de compactare, expandare si asociere asupra datelor
- Este unitate de prelucrare: in virgula mobila si zecimala
- Sunt module optionale, de sine statatoare, independente de PC
- Daca avem mai multe UAL este superscalar, daca avem mai multe unitati de comanda este o structura de hyper-threading
- Atribute: functie, tip implementare, structuri de date, operatii
- UAL e evoluat catre SIMD - unitati vectoriale care suprapun mai multe UAL-uri
- In timp UAL-ul s-a extins catre coprocesorul matematic

→ T - terminalul
- Asigura conversia din punct de vedere fizic a datelor
  - adaptare electrica
  - asigura sincronizarea intre terminal si UC de prelucrare sau PI/E care se ocupa de acel terminal
- Avem fie o singura componenta, fie 3(T, unitatea de comanda a T, switch T)
- Atribute: functie, tip cuplare, caracteristici

Exemple de Directive PMS:
→ Arhitectura Von Neumann
→ Sisteme cu mai multe procesoare si memorii (cu switch intre ele si terminale)
- S este cel mai important element pentru ca asigura conexiunile

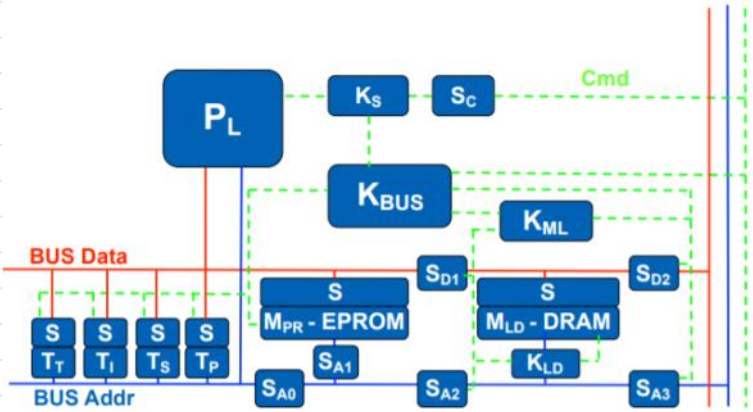
→ Sisteme Multiprocesor
- Vrem sa separam zona de procesare intensiva de zonele care au interactiune cu utilizatorul
- Exista retele dedicate intre procesoare printr-un switch separat
- Memoriile sunt partajate de procesoare

SBC (Single Board Computers)
→ Precursoarele System on a Chip
→ Are procesor, memorie + interfete I/O
→ Conecteaza pe o magistrala comenzi, date si adrese bidirectional
→ Resurse SBC:
- Private: procesor, EPROM, Interfete (S/P), RT Clock, Sistem de Intreruperi
- Locale: Memoria locala
- Globale: RAM (in spatiul de adresare)



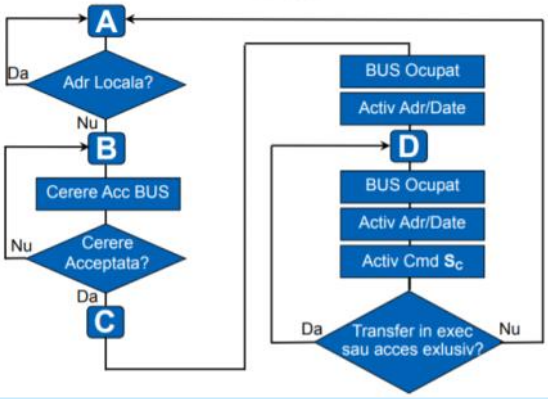
→ Pentru a gestiona accesul la magistrala (exclusiv) sunt necesare:
- KBUS - unitate de comanda pt accesul la MAG
- KML - unitate de control al accesului la memoria locala
- KLD - unitate de comanda a memoriei locale (refresh/etc)
- Ks - unitate de comanda a intregului sistem – asigura controlul procesoarelor la resursele sistemului

→ Fiecare procesor al SBC-ului vede memoria locala incapand de la 0 si memoriile celorlalte SBC-uri in spatii adiacente superioare
→ Structura PMS a SBC



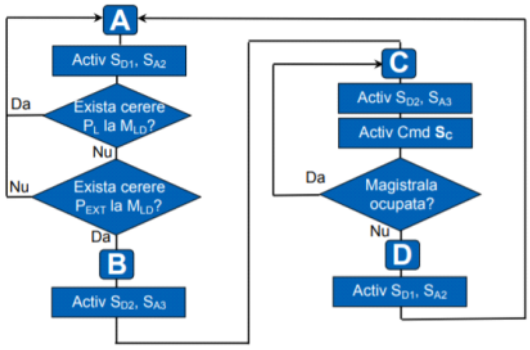
- Kbus este cel mai important pentru ca el controleaza interactiunea dintre magistrale si ce face SBC-ul local
- Pentru functionare este nevoie de mai multe interfete(Tt, Ti, Ts, Tp)
- Exista o memorie locala ROM (BIOS-ul SBC) si switchuri care fac separari in magistrala de adrese
- Exista o memorie RAM unde sunt datele
- Kml deserveste cereri la memoria locala in 2 situatii: PL doreste date sau un alt procesor extern vrea ceva
- Pt a separa magistrala de sistem avem switchuri (SAx - adrese, SDx - date, Sc - comenzi)

KBUS



- E nevoie de starea D deoarece dureaza pana se activeaza adr/date si sa nu fie gata cand se da comanda. Starea D introduce o latenta (un wait).

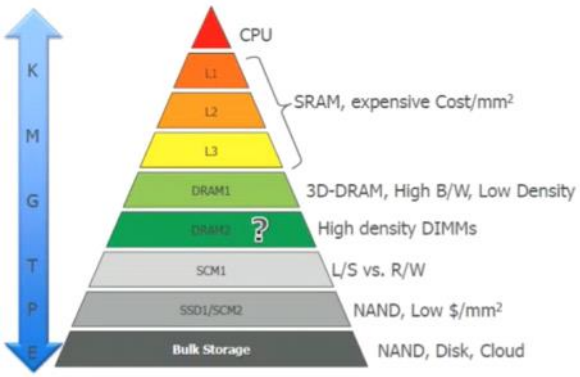
KML



Ierarhii de Memorii

Sunday, June 6, 2021 1:33 PM

Ierahii de memorii - Bottleneck-ul SC



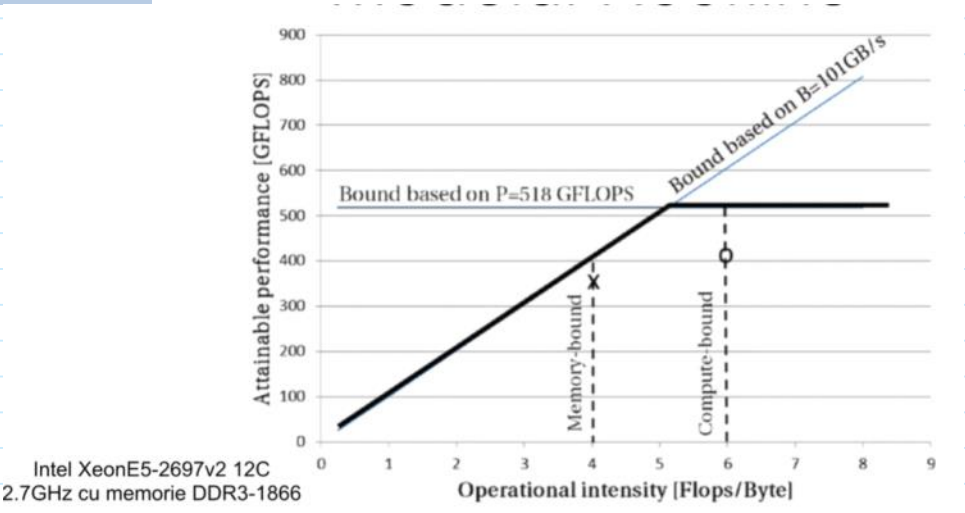




	Size (Byte)	Energy (pJ)	Delay (cycles)	Bandwidth (GB/s)
Reg	1K	10	1	1000
L1	32K	20	5	100
L2	256K	100	10	100
L3	8M	200	50	100
Off-chip	4G	2000	100	10

→ Memoria are o performanta cu mult mai mica decat procesorul.
→ Problema nu mai este doar despre frecventa, ci este despre cate unitati de procesare avem pt memoria data
→ Avansarile in memorii sunt mult mai slabe decat avansarea in procesoare

Modelul Roofline

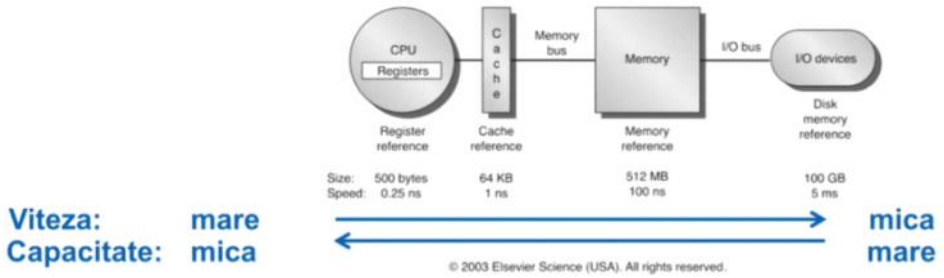


→ Modelul Roofline ne spune ca performanta unei procesor (in GFLOPS), adica ce poate sa dea UAL-ul, nu poate trece de o anumita limita roofline (P=518GFLOPS pe Intel Xeon….).
→ Pentru procesoare cu frecventa mai mare sau daca se face overclocking se poate obtine un roofline mai sus.
→ Memoria are un bandwidth limitat (B=101GB/s pt DDR3-1866).
→ Operation Intensity inseamna cate operatii fac per cate date am nevoie pentru a face operatia respectiva (suma in bytes a operantilor)

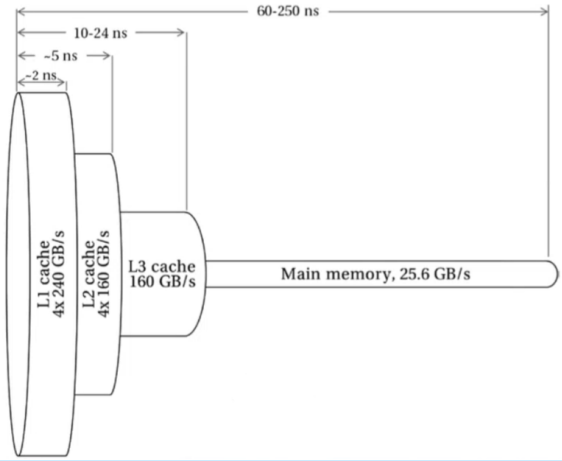
→ Daca OI = 4 suntem limitati de memorie (de bandwidth), trebuie sa ai minim 6 pentru a fi limitat de procesor.
→ Majoritatea codurilor sunt memory-bound, deci investitia in procesoare cu frecventa mai mare este proasta. Ar trebui sa scriem coduri care sa fie cat mai in zona de CPU-bound

Ierarhia de memorii

→ Deoarece nu putem realiza memorii suficient de rapide(ieftin), am dezvoltat ierarhia de memorii:
- L1 Cache (pe chip)
- L2 Cache
- L3 Cache (aproape intotdeauna)
- Memoria Principala
- Discurile fixe



→ Localitate spatiala – datele adiacente unor date utilizate sunt refolosite
→ Localitate temporala – datele sunt refolosite des



→ Fiecare tip de memorie are un bandwidth si latenta diferita
→ La trecerea intre generatii de procesoare este un salt al bandwidth-ului (de 20% in general) adica cresc mai mult unitatile de procesare noi decat iti poate oferi memoria

Localitatea

→ Programele acceseaza date din memorie apropriate intre ele prin instructiuni in Instruction Stream
→ Spatiala
- cand se acceseaza o locatie anume de memorie, este foarte probabil ca urmatorul acces sa fie in apropierea acestei locatii in memorie
- ideea e sa aduci aproape de procesor blocurile de date care sa contina datele utilizate (de obicei in cache)

→ Temporala
- cand se acceseaza o locatie anume de memorie, este foarte probabil ca aceasta locatie sa mai fie accesata din nou (deci sa-l pastrez intr-un registru)
- tinerea datelor accesate de curand, aproape de procesor, pentru ca instructiunile urmatoare sa le gaseasca disponibile
- ar trebui sa am multi registrii generali mari (in GPU se intampla)

→ Localitatea poate fi decisa de compilator si sistem de operare
→ Localitatea este exploatata de catre caching
→ Cand o locatie de memorie este accesata de catre procesor, se verifica urmatoarele in ordine: L1, L2, (L3), memoria principala, memoria virtuala
→ Acest proces are loc deoarece nu putem construi memorii suficient de mari si rapide la un pret rezonabil
→ Daca vom identifica multe din locatiile de memorie cautate in cache-ul L1, vom avea "iluzia" unei memorii mari si rapide, pentru o fractiune din cost

CACHE-ul

→ Cache-ul reprezinta locatia in care procesorul poate gasi date cautate mai aproape de el decat in memoria principala a sistemului
→ Un cache hit este momentul in care datele au fost gasite intr-un nivel de Cache
→ Un cache miss este momentul in care datele nu au fost gasite

→ Cand se inregistreaza un miss, un bloc este adus in Cache. Blocul contine celulele cerute, si aditional alte celule ce speram ca vor fi folosite de alte instructiuni (localitate spatiala)
→ Un cache miss determina un overhead seminificativ de lat + S/bw
  ▪ lat - latenta, bw - latimea de banda, s - dinemsiunea blocului in bytes
→ Cache-ul este compus din seturi de blocuri: Block Frames
→ Hit time este timpul de accesare al Cache-ului
→ Miss penalty este timpul necesar mutarii datelor pe diferite nivele de Cache catre procesor
→ Hit Ratio este procentul din timp in care datele sunt gasite in Cache
→ Miss Ratio este 1 – Hit Ratio
▪ Instruction Cache este Cache-ul ce contine cod - doar la nivel de L1
▪ Data Cache este Cache-ul ce contine date
▪ Unified Cache este Cache-ul ce contine atat date cat si instructiuni

Memorie Virtuala

→ Daca sistemul de calcul permite memoriei principale sa fie utilizata ca un cache pentru discurile fixe
→ Blocurile memoriei sunt numite pagini
→ Cand procesorul acceseaza o adresa dintr-o pagina ce nu se afla in memorie, se genereaza un page fault similar unui cache miss
→ Un page fault este extrem de costisitor (latenta lat e mare si largimea de banda bw este mica) si procesorul executa in mod normal alte taskuri in acest timp. Aceste page fault-uri sunt realizate de catre sistemele de operare.

Caracteristici Topic ale Memoriilor

	Dimensiune	Latenta (ns)	Largime de banda (MB/sec)	Gestionat de
Registre	< 1KB	0.25 - 0.5	25,000 - 100,000	Compilator
Cache	< 16MB	0.5 (on-chip) 25 (off-chip)	7500 -25,000	Hardware
Memoria Principala	< 16GB	25 - 250	1000 - 10,000	O/S
Discurile fixe	> 100GB	1,000 - 5,000,000	20 - 150	O/S

→ Programatorul poate sa determine structuri ale codului, flaguri de compilare si hint-uri catre

SO a.i. sa facem mai eficienta rularea pe o anumita arhitectura

Probleme in Designul Cache-ului

→ Ce dimensiune trebuie sa aiba un bloc de Cache?
- mare: miss penalty ridicat
- mica: oportunitate scazuta de exploatare a localitatii spatiale
- trebuie determinat dupa un benchmarking riguros si numeroase simulari
- trebuie sa fie eficient pentru aplicatii "tipice"

→ Patru intrebari necesare in designul unui cache (L1)
- Unde trebuie sa fie scrisa un bloc in cache? – block placement
- Cum se gaseste o linie in cache? – block identification
- Ce linie trebuie sa fie inlocuita in cazul unui cache miss? – block replacement
- Ce se intampla cand se scrie in cache? – write strategy

→ Limitari – design-ul trebuie sa fie simplu, astfel incat cache-ul sa fie rapid!

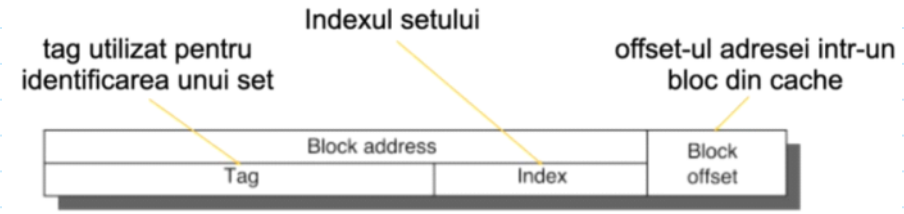
1. Block placement - Unde trebuie sa fie scrisa un bloc in cache?

→ Daca un bloc poate fi pozitionat doar intr-o singura pozitie in cache: mapare directa
→ Daca un bloc poate fi asezat oriunde: mapare total asociativa
→ Daca un bloc poate fi asezat intr-un subset de pozitii: mapare n set asociativa
→ Maparea directa este in fapt o mapare 1 set asociativa

→ Maparea n-way set associativa devine din ce in ce mai dificila si costisitoare de implementat pentru un n mare
→ Majoritatea cache-urilor sunt 1, 2 sau cel mult 4 set asociative
→ Cu cat n-ul este mai mare, cu atat este mai mica probabilitatea de aparitie a fenomenului de cache-thrashing (momentul in care doua (sau mai multe) regiuni de memorie sunt accesate in mod repetat si pot incapea impreuna in acelasi bloc din cache)

2. Block identification - Cum se gaseste o linie in cache?

→ Din adresa se poate identifica locatia acesteia in cache
→ Acest lucru se realizeaza impartind adresa in trei parti distincte:



Cache Mapat Direct
- Fiecare bloc de memorie/cache contine 2^b cuvinte
- Memoria Cache are contine 2^s linii
- Dimensiunea cache-ului este de 2^{s+b} cuvinte
- Fiecare linie din cache e identificata de tag
- Avantaje
  - design simplu ▪ Doar cateva comparatii intre parti din adrese de memorie
  - rapid, necesita putin hardware
- Dezavantaj major
  - este vulnerabil la thrashing (ar fi bun doar daca as citi/scrie doar din anumite linii)

Cache Mapat Set Asociativ
- Fiecare "set" din cache poate pastra mai multe linii din memorie: 2 - 8
- O linie de memorie poate fi mapata pe oricare dintre aceste doua pozitii
- Toate adresele cu un set index similar "concureaza" pentru doua frame bloc-uri posibile
- Este o solutie intre a face padding de mana si a lasa mia multe seturi

- Incearca sa rezolve problema de trashing

Cache Mapat Full Asociativ
- O linie de memorie poate fi plasata oriunde in Cache
- Se obtine considerabil mai putin thrashing
- Complexitate crescuta a unitatii de comanda (hit-ul costa mult)

3. Block Replacement - Ce linie trebuie sa fie inlocuita in cazul unui cache miss?

→ Cand are loc un miss, controlerul de cache trebuie sa inlocuiasca ceva
→ Pe un cache mapat direct, este foarte simplu – se va suprascrie continutul unui block-frame cu date noi aduse din memorie
→ Intr-o implementare n-set-asociativa insa, trebuie sa facem o alegere: care dintre cele n blockframe-uri trebuie suprascrise?
- Random - usor de implementat
- First-In-First-Out (FIFO) - aduce beneficii la cache-uri mai mici
- Least-Recently Used (LRU) - aduce beneficii la cache-uri mai mici, cel mai dificil de implementat

→ Trebuie mereu sa avem in vedere costul cache vs. latenta

4. Write Strategy - Ce se intampla cand se scrie in cache?

→ Trebuie sa stim care sunt proportiile de read/write
→ In general sunt mai multe citiri decat scrieri in cache (4:1)
→ Procentul de accese de tip scriere este ~ 7%, iar procentul de accese la date in memorie pentru scriere este ~ 21%
→ Designul cache-urilor a fost optimizat pentru a face citirile rapide, si nu scrierile (Make the common case fast!)
→ Procesorul trebuie sa astepte mereu la citire, si aproape niciodata la scriere!
→ Cand se face scriere, procesorul ii spune componentei de sub el (controller de cache, controller de memorie) sa se ocupe de coerenta datelor, adica sa le scrie si sa se asigure ca le vede cine trebuie
→ Daca scrierile sunt extrem de incete, legea lui Amdahl ne spune ca performantele globale ale sistemului vor fi scazute

Write-through
- Datele sunt scrise in acelasi timp in blocul din cache si in blocul din memoria principala (sau next level memory)
- Memoria si cache-ul sunt astfel mereu consistente

Write-back
- Se scrie doar in cache
- Datele sunt scrise in memoria principala doar cand un bloc frame este eliberat/inlocuit din cache
- Se foloseste bit "dirty" care indica daca blocul de inlocuit a fost modificat
- Memoria si cache-ul nu mai sunt totdeauna consistente!

Cache Trade-Offs

→ Write-Through
- Mult mai usor de implementat ca Write-Back
- Nivelul urmator de cache va avea intotdeauna o copie actualizata a datelor utilizate
- Foarte important pentru sisteme multiprocesor – simplifica problemele de coerenta a datelor

→ Write-Back
- Rapid – scrierile se petrec la viteza cache-ului si NU a memorie
- Rapid – actualizari multiple ale aceluiasi bloc de cache vor fi scrise inapoi in memoria principala in seturi de dimensiuni mai mari, utilizand mai putin din latimea de banda a

memoriei
- Solutie buna pentru servere cu multe procesoare/core-uri

→ Cand doua linii de memorie sunt in cache si a treia linie vine, una din primele doua linii trebuie sa fie inlocuita. Decidem care din ele trebuie inlocuita utilizand euristici (LRU, FIFO, Random)

→ In general, cache-urile sunt asociative (pot preveni trashing, necesita mai mult hardware decat maparea directa si sunt mai incete si mai scumpe)

→ Write-back se pune mai aprope de procesoare (L1) si write-through in zone mediane

Imbunatatirea performantelor

→ Hardware
- Reducerea penalitatilor unui Cache Miss
- Reducerea ratei de Cache Miss-uri
- Reducerea timpului de rezolvare a unui Hit
- Imbunatatirea memoriei (performanta crescuta si cost redus)

→ Optimizari de Compilator
- Sunt mai multe moduri in care un cod poate fi modificat pentru a genera mai putine miss-uri: compilatorul si utilizatorul
- Un compilator bun trebuie (in principiu) sa poata optimiza
- Cateodata insa, compilatorul nu poate face tot ce este necesar

→ Memoria 1D este descrisa de un singur numar: adresa de memorie. Astfel, este necesara o mapare de la 2D la 1D

→ Sunt posibile n^2 mapari, insa se folosesc doar row-major si column-major (Fortran)

→ Liniile de memorie/cache sunt si ele puse una dupa alta (ca in row-major). Memoria este insa mai mare decat cache-ul si doar o parte din ele intra in cache

→ Vectori alocati static
- Pentru parcurgerea unei matrici, ordinea i-j este mai buna (se genereaza un miss cand se incepe o linie noua, insa in rest e hit)
- La dimensiuni mici ale matricei performantele sunt asemanantoare pt ca toate datele au loc in cache
- ordinea i-j => miss_rate = 6.25% (e/b)
- ordinea j-i => miss_rate = 100%

→ Vectori alocati dinamic
- La alocarea dinamica a unui vector, in Fortran se poate aloca cu dimensiuni variabile, insa in C trebuie sa alocam explicit memoria ca un vector de vectori
- La alocarea dinamica se obtine o localitate mai buna, indiferent daca e column sau row-major
- Vectorii dinamici sunt buni cand avem volum mare de date

→ Liste Inlantuite
- Fiecare element al listei va fi alocat dinamic la momentul inserarii, deci elementele listei nu vor fi contigue
- Parcurgerea listei in ordine va genera in “general” cate un cache-miss pentru fiecare element!
- Acest lucru poate pune probleme majore de performanta daca lista este suficient de lunga si ea este parcursa des
- Putem avea o localizare mai buna a detelor din liste prin implementarea listelor ca vectori (next se realizeaza imediat si mai des, insa insert si delete mai greu)

- In C, putem utiliza “realloc” pentru a minimiza copierea elementelor
- Operatia next este insa aproape instantanee
- Utilizatorul trebuie astfel sa identifice “the common case” si sa selecteze implementarea corespunzatoare

→ Masurarea L-ului
- Cea mai buna performanta: pas=1
- Cea mai proasta performanta: pas ≥ L
- Daca masuram performanta codului pentru diverse valori ale pasului, obtinem un grafic
- Pragul (pasul in bytes) este dimensiunea liniei de cache

→ Masurarea Dimensiunii Cache-ului
- Daca e mai mare de 1024 de bytes nu vom avea miss-uri, daca e mai mic de 1024 va fi un nr de miss-uri

Concluzii
→ Exista 2 sau 3 nivele de cache
→ Cache-urile aproape de procesor sunt in general mapate direct, si cele mai departate sunt asociative
→ Cache-uri diferite de date/instructiuni aproape de procesor, si unificate in rest
→ Write-through si write-back sunt la fel de des intalnite, dar nu va exista nicidodata o implementare write-through pana la memoria principala
→ Liniile de cache aveau in mod normal 32-byte, dar acum exista foarte des linii de 64 si 128
→ Cache-uri neblocante
- La un miss, nu bloca procesorul ci executa instructiuni de dupa load/store-ul curent
- Blocheaza procesorul doar daca aceste operatii utilizeaza date din load
- Cache-urile trebuie sa poata sa serveasca multiple accese “invechite"

Performanta Cache-urilor
→ Performanta cache-ului este data de accesul mediu la memorie
Timpul de acces la memorie = hit time + miss rate * miss penalty
→ Miss penalty depinde de tehnologia de implementare a memoriei
→ Cand incercati sa estimati performantele unui sistem de calcul, comportamentul cache-urile trebuie luat in considerare (Amdahl: CPU vs. Memorie)
→ Un procesor mai rapid e “lovit” mai tare de memorii lente si miss-rate-uri crescute
→ Desi unele structuri de date par naturale, ele pot fi extrem de ineficiente in cache

Taxonomii si Utilizarea SC

Monday, June 7, 2021 12:16 PM

Paralelism

→ Paralelism la nivel de
- Bit (Bit-level parallelism) - Operatii in virgula mobila
- Instructiune (ILP)
  - Mai multe instructiuni pe ciclu
  - Instructiunile se pot planifica a.i. sa folosesti cat mai multe operatii din subsisteme (ex. cand UAL face ceva sa ceri date de la memorie folosind controller-ul DMA)
- Memorie - Suprapunerea intre operatii cu memoria si de calcul
- SO - Mai multe thread-uri, procese pe CPU-uri multiple, in cadrul aceleasi masini

→ Paralelism distribuit - Mai multe masini conectate impreuna (sau MPI)

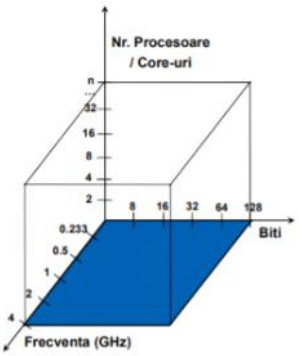
Aplicatii Paralele

→ High-Energy Physics
- teoria fundamentala a particulelor elementare
- generatorii cei mai mari de date de calcul computational (date de ordinea PentaB/ms)

→ Simulari nucleare, Dinamica fluidelor(avioane, navete)

→ Recunoastere in timp real al vorbiri, Sisteme grafice de animatie, Sisteme de navigatie distribuite, Biochimie, Astrofizica, Geofizica, Meteorologie

Structuri de Calcul cu Prelucrare Paralela



→ Aria - puterea de calcul (procesarea pe un singur core)
→ Volumul - marimea puterii de calcul
→ **n** e limitat "doar" de cost - Arhitecturi paralele de calcul

→ Orice problema are un grad de paralelism intrinsec ce depinde de natura ei
→ Gradul general de paralelism al codurilor este ~ 10%
→ La un grad asa mic de paralelism nu mai scaleaza programele
→ Daca avem un grad de paralelizare de 90% nu este necesar sa folosim mai mult de 11 core-uri
→ Majoritatea codurilor nu are sens sa le paralelizezi(ca sa aiba sens trebuie sa ai 90-95%)
→ IBM BlueGene/L
- supercalculator de 131,072 procesoare care ruleaza in paralel cu memorie aproape la viteza procesorului
- Format din rack-uri si se pune accent pe switch-uri

Nivelul Prelucrarii Paralele

1. Paralelismul la nivel de Bloc/Job

→ Intre Job-uri diverse: mecanism de salvare a contextului; un ceas de timp; canale I/O

→ Intre fazele unui Job
  - citirea sursei programului, compilare, link-are, executarea codului obiect, salvarea rezultatelor

→ Un program face multi-tasking cand isi imparte job-urile la mai multe unitati
→ Anumite faze ale unui job pot fi suprapuse

2. Paralelismul la nivel de subansamble Hardware

→ Intre elemente de prelucrare ale vectorilor
→ Intre componentele logice ale dispozitivelor aritmetico/logice (e.g. carry look ahead)

3. Paralelismul la nivel de Program

→ Intre diverse sectiuni (independente) ale unui program
→ Intre buclele (loop-urile) unui program
→ Ambele presupun analiza dependentelor de date
→ Daca programatorul specifica ca o sectiune e paralela, dar compilatorul descopera o dependenta de date da warning (daca lasam acest lucru in seama compilatorului)
→ In sisteme mai avansate exista analiza la RamTime

4. Paralelismul la nivel de Instructiune

→ Intre diverse faze ale ciclului instructiune
  - seriala, serie-paralela, paralela

→ Este necesar un mecanism de predictie al salturilor(branch prediction)

Clasificarea Sistemelor de Calcul

→ Sisteme Matriceale
  - Fiecare element de procesat e practic un mini-core
  - Sunt precursoarele GPU-urilor din ziua de azi
  - In loc sa functioneze cu un scalar, unitatea de baza este vectorul si fiecare instructiune se face pe intreg vectorul
  - Exista legatura intre o celula si toti vecinii, deci trebuie gandit altfel load-ul si comunicarile cu vecinii
  - Se utilizeaza la senzori

→ Sisteme Multiprocesor
  - Formate din multe unitati de prelucrare interconectate printr-o retea de comutare (slab cuplata)
  - Fiecare unitate de prelucrare are o memorie proprie
  - Sistemele lucreaza independent la realizarea aceluiasi Job

→ Sisteme Pipeline
  - Unitati de prelucrare asezate in banda de asamblare (RISC)
  - Fiecare UC executa o prelucrare specifica si transfera rezultatul subansamblului adiacent

Taxonomia lui FLYNN

→ impartire in functie de fluxul de instructiuni(FI) si fluxul de date(FD)
- SISD = Single Instruction Single Data Stream (structura Von Neumann)
- SIMD = Single Instruction Multiple Data Stream
- MISD = Multiple Instruction Single Data Stream
- MIMD = Multiple Instruction Multiple Data Stream

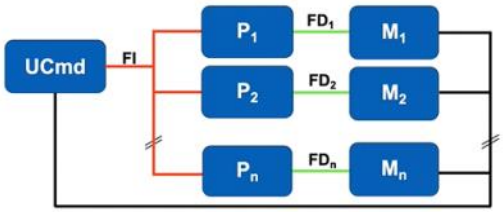
1. SISD

→ calculatorul clasic one-core (nu are arhitectura paralela)
→ exista un FI si un FD



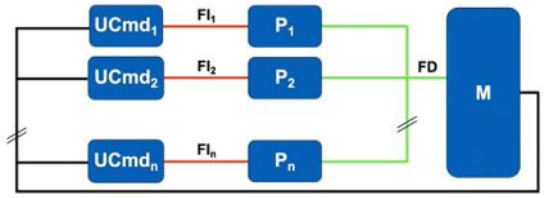
2. SIMD

→ sunt masini vectoriale cu o singura Ucmd si mai multe procesoare si memorii
→ exista un FI si n FD
→ Ucmd are legatura si la memorii
→ sincronizarea se face la nivel de instructiune
→ daca cumva datele nu sunt disponibile in vreuna dintre memorii, instructiunea nu se poate executa pentru niciuna (daca un P nu are date este tot afectat deci trebuie sa asezam datele intr-un anumit fel)
→ Toate procesoarele fac aceeasi operatie impusa de UCmd prin FI



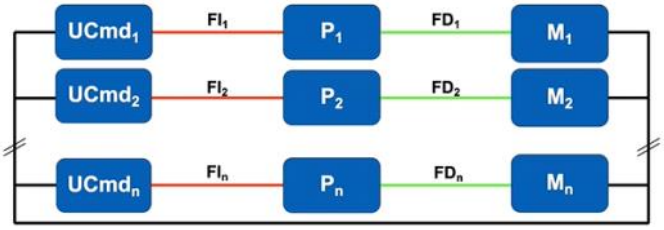
3. MISD

→ au mai multe Ucmd si procesoare, dar o singura memorie
→ exista n FI si un FD
→ domeniu de aplicatie restrans si special (aplicatii meteo, evidenta populatiei)



4. MIMD

→ masini paralele clasice
→ pot comunica procesoarele intre ele prin intermediul unui switch
→ pot comunica procesoarele cu memoriile
→ exista n FI si n FD
→ toate procesoarele participa la acelasi program
→ mod programare:
  - Shared memory (strans cuplate) - Memorie Partajata (e.g. OpenMP)
    - Programarea este transparenta, procesoarele vorbesc intre ele (dar costa)
  - Distributed memory (slab cuplate) – Transfer de Mesaje (e.g. MPI)
    - Vorbeste oricine cu oricine



Exemplu de Utilizare - Matrix Multiply

1. SISD - O(n3) - complexitate nesatisfacatoare
2. SIMD

- Avem n procesoare care executa aceeasi instructiune odata
- In fiecare calcul se calculeaza cate o linie si nu doar un element
- Toata bucla k se executa pe toate unitatile deodata => raman 2 for-uri
- O(n2) - considerabil mai buna

```
for i = 0 to n - 1
   cik = 0 (0 ≤ k ≤ n - 1)
   for j = 0 to n - 1
      cik = cik + aij * bjk (0 ≤ k ≤ n - 1)
   end j loop
end i loop
```

- Fiecare element al matricei produs C este o suma ce se efectueaza secvential
- aij nu depinde de k, deci nu se calculeaza complet paralel
- PROBLEMA: bjk se afla in toate memoriile, dar aij se afla doar un una dintre ele
- SOLUTIE: o instructiune de BROADCAST si o retea de comutare(RC)
- Deci este nevoie de hardware aditional (partea de interconnect) si broadcast
- Pj citeste aij prin RC, care e difuzata catre toate procesoarele prin Broadcast
- Sunt probleme semnificative la comunicatiile inter-procesoare, la accesul si organizarea datelor

3. MIMD
   - Aceeasi problema ca la SIMD
   - O(n2) - ca la SIMD
   - O unitate-comanda/procesor trebuie sa preia functia de master: organizare si control + partajarea calculelor pe procesoare individuale

```
for k = 0 to n - 2          // nu si pentru el insusi
   FORK Adr
end k loop
Adr:
   for i = 0 to n - 1
      cik = 0 (0 ≤ k ≤ n - 1) // pe exact k coloane
      for j = 0 to n - 1
         cik = cik + aij * bjk (0 ≤ k ≤ n - 1)
      end j loop
   end i loop
   JOIN
```

   - Restul procesoarelor se numesc diretive de FORK si JOIN
     - FORK: desface un FI in n FI executabile simultan pe procesoare independente
     - JOIN: reuneste n FI intr-unul singur cand cele n FI s-au terminat
   - FORK & JOIN:
     - Pentru fiecare FI exista un contor unic ce e initializat cu 0 la inceput
     - Pentru fiecare FI contorul este incrementat si comparat cu n
     - Trebuie ca toate procesele sa se fi terminat pana la n si abia apoi se continua cu urmatoarea secventa liniara pe master
4. SIMD vs MIMD
   - In SIMD procesoarele se sincronizau la nivel de instructiune
   - In MIMD nu exista (neaparat) sincronizari intre FI ale procesoarelor
   - La SIMD se calculeaza elementele lui C pe linie si coloana (FI unic)
   - La MIMD orice procesor poate calcula orice element din C (FI-uri diferite)

   - In mod deliberat programul pt SIMD a fost scris a.i. actiunile P-urilor sa simuleze actiunile din structura MIMD
   - Castigul e acelasi
   - MIMD e mai usor de programat & utilizat decat SIMD, dar mai scump (procesoarele din MIMD sunt mai performante si au un overhead pe partea de gestionare)

Exemplu fara Dependente de Date

→ Avem o coada de asteptare cu procese
→ Procesoarele isi fac task-urile pana dau join.
→ Masterul poate si el sa ia un task daca nu face nimic la momentul acela

→ Se foloseste cand avem mai multe subrutine care nu au dependinte intre ele

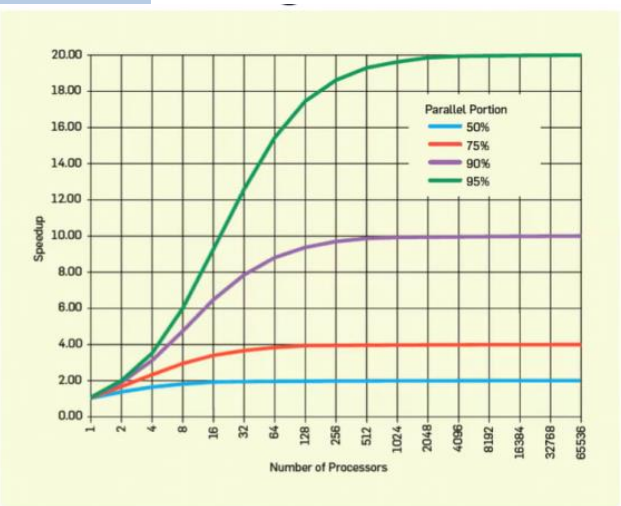
Exemplu cu Dependente de Date

→ Exista un arbore de dependinte (ex: P3, P4 nu se poate executa pana nu e gata P2)
→ Primul proces (masterul) formeaza coada si vede care proces depinde de care
→ Baga procesele in coada daca s-a terminat dependinta lor (daca exista)
→ Exista procesoare ce asteapta datorita dependentei de date
→ Mai exista probleme de resurse (sa se bata pe aceeasi resursa sau sa nu fie disponibila)

Concluzii Dependinte de Date

→ Alocarea dinamica este mai buna
→ FIFO nu e prea buna
→ Programe de uz general → 10% paralelism
→ Programe dedicate → 99+% paralelism
→ Dependente de date introduc sicronizari in timp, deci scad eficienta codurilor
→ Eficienta MIMD depinde de:
  - Eficienta algoritmilor de planificare
  - Consumul de timp impus de precedenta de date
→ Problema repartizarii pe procesoare este NP completa(nu se poate face mai bine decat ceea ce exista)

Legea lui Amdahl



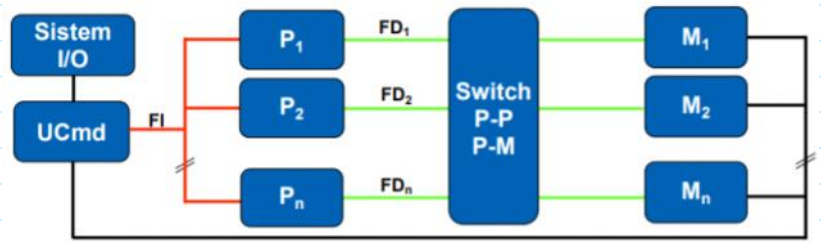
→ Obtinem un speedup de maxim 2 pt 50% paralelizare
→ Daca vrem sa profitam de arhitecturile moderne trebuie sa folosim paralelism la maxim

Arhitecturi SIMD

Monday, June 7, 2021 6:25 PM

Structura SIMD

→ Sunt adecvate prelucrarilor vectoriale
→ O singura Ucmd care genereaza un singur flux de instructiuni care se duce spre UAL-uri
→ Mai multe procesoare aritmetice si module de memorie
→ Retea de comutatie P-P & P-M (prin switch-uri)



→ Pentru aplicatii neorientate pe aplicatii vectoriale sunt ineficiente

Componente SIMD

1. Ucmd
   - Citeste, interpreteaza si executa instructiuni de control
   - Necesita registre generale, unitate de prelucrare si memorie locala
   - Executa si instructiunile de ramificatie (nu ca Pi)
2. Pi
   - Sunt sincronizate la nivel de instructiune
   - Fiecare procesor lucreaza pe un alt set de date
   - Trebuie sa aiba registre locale (UAL)
3. M
   - Formata din n module
   - Contine instructiuni (de comanda & prelucrare) si date
4. Instructiuni
   - De control → executate pe Ucmd
   - De prelucrare vectoriala → executate de Pi
   - Fluxul de instructiuni e similar cu cel din sisteme secventiale
5. Registrii index
   - Globali (in Ucmd)
     - afecteaza cu aceeasi cantitate adresa operanzilor
   - Locali (in fiecare Pi)
     - pot fi independente si se refera la zona de memorie Mi asociata fiecarui procesor
     - Daca un Pi trebuie sa ia date de altundeva decat din Mi trebuie sa discute prima data cu switch-ul
     - La cereri multiple din aceeasi zona se poate ajunge la secventialitatea accesului la date!

Exemplu - Matrix Multiply

→ Toata bucla k se executa pe toate unitatile deodata => raman 2 for-uri
→ Elementele unei linii sunt in module diferite de memorie
→ aij nu depinde de k, deci nu se calculeaza complet paralel
→ Ne trebuie Broadcast pentru aij sau sa secventializam
→ Linia rezultat: ci1, ci2 … cin este calculata in paralel pe P1, P2 … Pn

Implementare Salturi Conditionate in SIMD

→ Avem doar un singur FI in care se face sincronizarea
→ In sistemele SIMD salturile conditionate se implementeaza prin divizarea fluxului de instructiuni (doar 1FI posibil!)
→ Un flux pentru conditia indeplinita
→ Alt flux pentru conditia neindeplinita
→ Cele doua FI se executa la momente diferite de timp si secvential! → paralelism n/2
→ Pentru a implementa este nevoie de resurse de mascare a activitatii procesoarelor



→ Procesoarele au toate niste registrii de masca
→ Cand merg pe if activez procesoarele pentru conditia indeplinita si procesez, celelalte sunt dezactivate

Organizare Date in SIMD

→ Probleme:
- Repartizarea in memoriile Mi & accesul paralel
- Citirea vectorilor intr-o singura instructiune
- La matrice: acces la linii si la coloane → necesara transpunerea / deplasarea datelor?

→ Memoria:
- Poate face cel mult un acces la un ciclu de lucru
- n operanzi in n module diferite pot fi cititi simultan
- Defavorabila plasarea in acelasi modul al operanzilor → citirea va fi secventiala in n cicli de acces!

1. Reprezentarea Directa pe Linii
   - Matrice A[n,n] & structura SIMD cu n procs si n memorii
   - Cu acces simultan la liniile matricei:
     - Acces paralel la linii
     - Acces secvential la coloane
   - LOAD A[i] - fiecare element al liniei se afla pe module diferite ale memoriei, deci se intampla intr-un singur ciclu de ceas
   - Daca vreau datele pe coloane ar fi secvential (load in sine e prost)
2. Reprezentarea Directa pe Coloane
   - Cu acces simultan la coloanele matricei:
     - Acces secvential la linii
     - Acces paralel la coloane
   - LOAD A[i] - coloane se incarca direct intr-un singur ciclu de ceas
   - Daca vreau datele pe linii ar fi secvential

! Reprezentarile directe pe linii/coloane sunt incompatibile la acelasi moment de timp

3. Preprezentare Deplasata Ciclica

→ Pentru o matrice 4x4 vreau sa inmultesc linia si coloana 3
→ Se schimba modul in care organizez datele in matrice
→ Se shifteaza dreapta fiecare linie cu n (n - liniei)
→ Cand se face acum load, elementele de inmultit se afla pe aceeasi linie



→ Este necesar un shuffling cu reteaua de comutatie (ai1 nu e pe P1) pentru ca se pierde coerenta datelor

Concluzii Organizarea Datelor

→ Programe simple → acces secvential la linii & coloane
→ Programe complicate → acces paralel la linii & coloane
→ Cu organizarea deplasata ciclica accesul la linii & coloane se face intr-un singur ciclu
- La linii – adresa fizica/actuala a liniei
- La coloane – sunt necesari registre index private in Pi

→ Costul platit paralelismului este consistenta datelor
→ Este nevoie de o retea de comutatie CROSS-BAR care sa permita trimitearea datelor P-P si P-M

Concluzii Algoritmi SIMD

→ Elaborarea unor algoritmi eficienti necesita
- Organizarea corespunzatoare a structurilor de date
- Resurse si Instructiuni in plus: Registre locale ale sistemului de calcul
- Existenta unei retele de comutatie P-P

→ Etapele elaborarii unui program sunt
- Descrierea algoritmului
- Paralelizarea algoritmului
- Optimizarea paralelizarii curente

Probleme de comunicatie in SIMD

→ Problema consta in miscarea datelor intre memorii a.i. procesoarele sa poata sa proceseze in acelasi timp datele (avand un singur FI)
→ Daca P-urile vor sa acceseze date in acelasi M in acelasi timp e o problema, e nevoie de serializare

→ Datele trebuiesc plasate in module distincte de memorie pentru cresterea vitezii
→ Datele trebuiesc permutate pentru a furniza operanzi pentru a utiliza eficient procesorul
→ Pentru rezolvarea celor doua probleme sunt necesare retele de comunicatie P-P si P-M

Retele de Comutatie - RC

→ RC P-P sunt implementate la nivelul procesoarelor
→ RC P-M pot fi implementate cu switch-uri Cross-Bar (fiecare cu fiecare)
- Pentru n procesoare sunt necesare n(n-1)/2 SP
- Cand n creste costul devine extrem de mare

→ In functie de aplicatie sunt necesare RC dedicate care ofera:
- fiabilitate, costuri reduse, viteza mare de comutatie/transfer, facilitati de intretinere

1. Interconexiunea Circulara

→ Deplasarea ciclica a datelor intre procesoare (P0 cu P1, P1 cu P2 …)
→ Conexiunile sunt permise numai cu vecinii
→ Nu exista conexiuni directe intre i si j

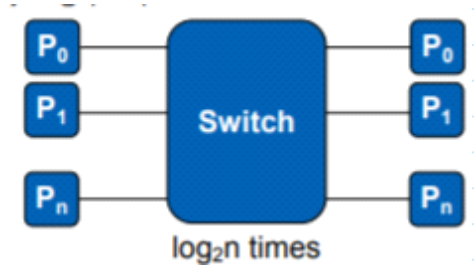
→ Aplicatii: ecuatii diferentiale, propagarea caldurii printr-o bara, meteo

2. Interconexiunea Matriceala

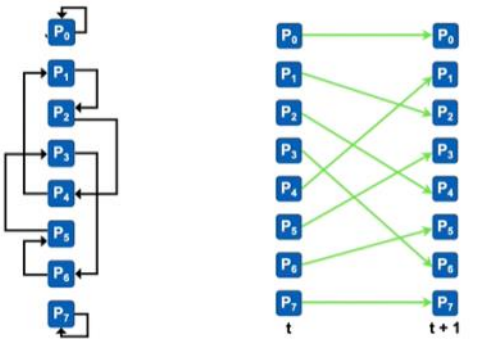
→ Asigura comunicarea cu patru procesoare vecine (sus, jos, stg, drp)
→ Se face update in functie de vecini
→ Majoritatea SIMD-urilor folosesc aceasta interconexiune (Tor)
→ Aplicatii: evolutia temperaturii in volum

3. Intercalarea Perfecta - Shuffle

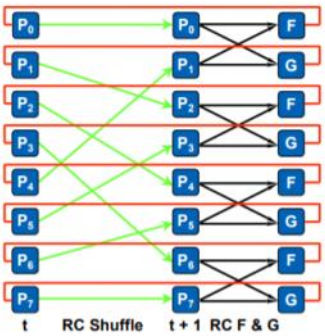
→ Interconectare intre retele LOCALE
→ Cum legam procesoarele intre ele a.i. sa isi paseze datele in timp util
→ Aplicatii: trans Fourier, sortari, transpuneri
→ Vrem sa modificam datele in zona de switch P-P
→ Asigura conectarea intre procesoarele unei structuri SIMD de n procesoare cu
  - n-2 comunicatii simultane (fara conexiunea cu el insusi)
  - dupa log2n iteratii prin switch datele se intorc pe procesorul sursa ca sa pastram coerenta datelor



→ Se fac niste permutari elementare



→ Adaugam procesoarelor si niste functii (doua RC):
  - Shuffle
  - Adiacenta (pt F si G) - F pe pare, G pe impare
→ Dupa log2n iteratii ajung inapoi pe proceasoarele sursa



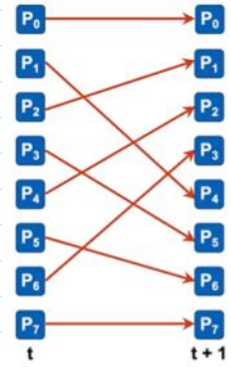

```
for i = 1 to log2n
   shuffle[y]
   Yi    = F[Yi,Yi+1]
   Yi+1  = G[Yi,Yi+1]
end i loop
```

→ F si G sunt niste functii care se aleg in functie de ce aplicatie vrem
→ Bun pentru aplicatii iterative

→ Probleme aplicatii iterative:
- se incarca inegal procesoarele (unele stau degeaba dupa prima iteratie)
- La fiecare pas trebuie schimbata RC in mod dinamic
- Solutia: se introduce comunicare liniara si RC statica si se obtin si sume partiale

4. Conectare Inversa - Inverse Shuffle

→ Inversul in oglinda al shuffle-ului
→ Se foloseste ca sa pastram reteaua de comutatie la fel si eficienta buna
→ Adecvat pentru operatii recurente, calculabile in log2n pasi
→ Se introduce si o masca care sa activeze/dezactiveze procesoarele
→ Legatura este identica pentru toate iteratiile, nu mai schimb reteaua deci e mai rapid




```
Yi = Ai  (0 ≤ i ≤ n-1)
for j = 1 to log2n
   Yj    = Yj + Yj-1
   invshuffle[Y], MASK
   compute MASK
end j loop
```

→ Se face o legatura directa si apoi un Inverse Shuffle
→ Trebuie transmise si mastile, care se modifica si ele la fiecare iteratie din Inverse Shuffle

Permutari Elementare

→ In retele putem avea permutari utile pentru o serie de aplicatii
→ Avem nevoie de o asezare diferita a datelor pentru a putea aranja datele P-P sau P-M ca sa putem face deplasarea organizata in SIMD
→ Specifica conexiunea intre n resurse ca o functie bijectiva pe o multime ordonata
→ Se pot realiza in mai multe feluri:
- Permutarea de baza
  - Bitul cel mai nesemnificativ devine cel mai semnificativ (invers lui Shuffle)
- Permutarea cu intercalare perfecta
  - Bitul cel mai semnificativ devine cel mai nesemnificativ
- Permutarea cu negare de bit – Negate Bit Permutation

→ Definirea acestor permutari se poate face pe baza reprezentarii binare a adreselor resurselor din multime

Concluzii Structuri SIMD

→ Probleme:
- Calculele trebuiesc descrise de iteratii de tip vector a.i. majoritatea timpului sa fie ocupat cu operatii cu caracter identic
- Operanzii (simultani) trebuiesc repartizati in module diferite de memorie
- Trebuiesc identificate conexiunile intre operanzi a.i. ele sa fie cat mai aproape de conexiunile fizice intre procesoare

→ Clasa problemelor SIMD nu este universala (probleme dedicate)
→ Algoritmii, sistemele de interconectare si structura datelor trebuiesc adaptate in consecinta

Retele de Comutare

Tuesday, June 8, 2021 12:59 PM

Comutatoare

1. Ierarhice (P-M) - conecteaza o componenta de tip acu una de tip



2. Neierarhice (P-P, M-M) - conecteaza 2 componente de tip a



Comutatoare IERARHICE

→ Comutatorul Simplu - Poarta

- Asigura comunicarea intre componente de tip a si b
- Descris ca S {poarta, 1a, 1b}: un switch poarta conecteaza a1 cu b1
- De mai multe tipuri in functie cum pui legatura (L) - comutarea e la elementul care nu are L:

S {poarta, 1a, 1b, comutare la b}



S {poarta, 1a, 1b, comutare la a}



S {poarta, 1a, 1b, cu legatura la a si b}



- Poate conecta o resursa la magistrala sau o magistrala la alta magistrala

→ Comutatoare Ierarhice Duplex

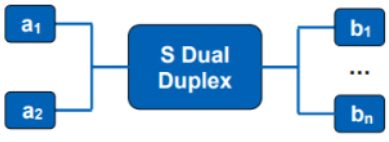
- Asigura comunicatia intre o componenta de tip a si mai multe componente de tip b
- Descris ca S {poarta, 1a, nb, concurenta 1, 1Sp}



- Se pot considera toate variantele de la comutatoarele simple

→ Comutatoare Ierarhice Dual Duplex

- Asigura comunicatia intre doua componente de tip a si mai multe componente de tip b
- Descris ca S {poarta, 2a, nb, concurenta 2, 2nSp}



- Switch-ul trebuie sa decida care dintre a-uri se conecteaza cu care dintre b-uri

→ Comutatoare Ierarhice Legatura Multipla

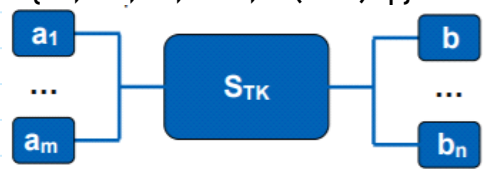
- Asigura comunicatia intre m componente de tip a si n componente de tip b
- Este cunoscut si ca Cross-Bar-Switch
- Descris ca S {CB, ma, nb, c = min(m, n), m*nSp}



- Problema e numarul de switch-uri poarta e m*n, deci e prohibitiv
- Orice a poate comunica cu orice b
- Pot avea loc transferuri multiple si simultane (concurenta de min(m, n))
- Este dificila implementarea unitatii de comanda pt Scb
- Se numeste un HOP trecerea de la un a la un b

→ Comutatoare Ierarhice Trunchi K

- Imbina conectivitatea oferita de legatura multipla cu costul redus al celorlalte tipuri de comutatoare
- Orice a **poate** comunica cu orice b
- Descris ca S {TK, ma, nb, c = k, k*(m+n)Sp}

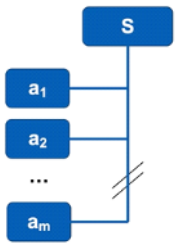


- Limiteaza numarul de conexiuni la k (nu min(n,m))
- In functie de m si n si de trafic, se va determina un k optim
- Putem avea un k cel mult egal cu nr de memorii

Comutatoare NEIERARHICE

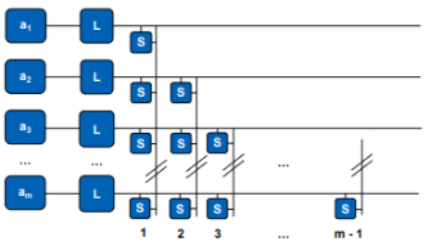
→ Comutatoare Duplex Neierarhice

- m elemente de tip a si conexiunea este cate una odata (principiul HUB)
- Este un Bus inlantuit
- Descris ca S{Dn; ma, c:1; mSp}



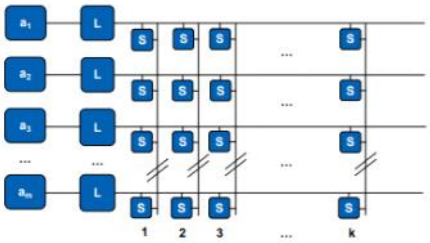
→ Comutatoare Neierarhice Legatura Multipla

- m elemente care pot sa comunice fiecare cu fiecare
- Conexiunile sunt in numar de m/2
- Descris ca S{lm-neierarhica; ma, c:m/2; m(m-1)/2 Sp}



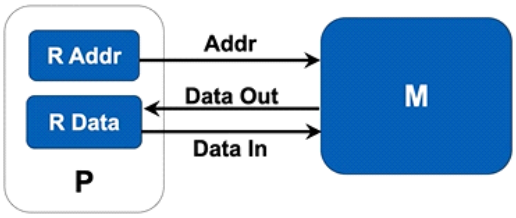
→ Comutatoare Neierarhice Trunchi-K

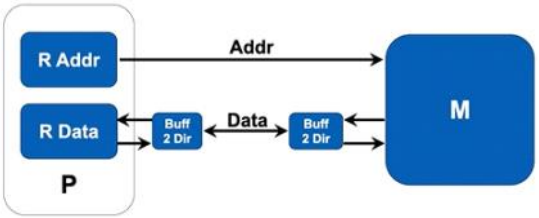
- Avem aceeasi structura de m elemente, dar de acasta data u se comunica cu alte m structuri, ci cu k
- Conectivitatea e min(k, m/2) - nu e prea bine, trebuie ales un k bun sa nu se ajunga la m/2
- Descris ca S{tk; ma, c:min(k,m/2); m*k Sp}
- Este mai complicata ca la legatura multipla



Comutatoare Procesor Memorie

→ Switch Processor Memory → SPM

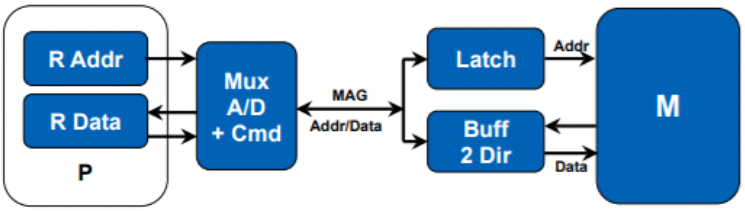




- Liniile de adrese si date sunt complet separate
- Nu avem niciun switch
- Costa la performanta
- Nu se justifica liniile

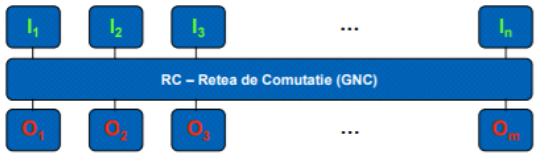
- Liniile de intrare/iesire sunt multiplexate prin magistrala de date
- Se introduc buffere bidirectionale
- Bufferele sunt fie MUX fie TS (Tree State)
- Ar fi bine sa punem si adresele pe magistrala

→ Varianta finala contine si adresele pe magistrala
→ Latch face decodificare, Bufferele sunt fie MUX fie TS (Tree State)
→ Astfel nu tinem procesorul la viteza memoriei si datele sunt coerente



Interconectarea in Sistemele cu Resurse Multiple

→ In structuri SIMD sau MIMD sunt necesare retele de comutare pentru a face legatura P-M
→ Realizarea interconectarii resurselor (P, M & I/O) se face printr-o retea de comutatie (RC)
→ Generalised Connection Network (GCN)
- Graf in care arcele conecteaza perechi de resurse

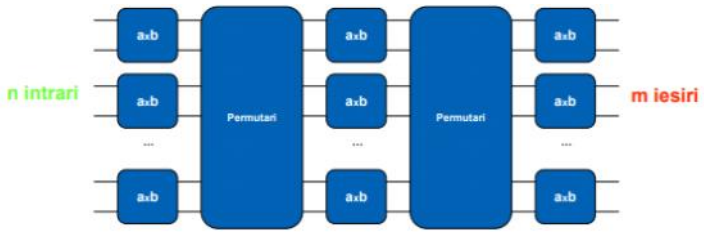


- Reguli RC:
  - O intrare poate fi conectata la orice iesire
  - O intrare poate fi conectata simultan la mai multe iesiri
  - O iesire poate fi conectata numai la o intrare
  - Concurenta maxima e data de min (m, n)
- Se fac n*m corespondente
- Caracteristici:
  1. Intarzierea de propagare (Δt) – timpul de raspuns ce depinde de numarul maxim de comutatoare elementare (de tip poarta) ce asigura legatura intre intrare si iesire
  2. Complexitatea (nC) – numarul total de comutatoare elementare utilizate in implementarea GCN-ului

Implementari ale RC (GCN)

1. Cross Bar
   - Comutare directa cu n·m comutatoare (Δt = 1)
   - Printr-o structura organizata pe un singur nivel in care se stabilesc legaturi directe intre intrari si iesiri
   - Unitatea de comanda trebuie sa rejecteze 2 I (intrari) / 1 O (iesire)
   - Greu de implementat si nu este flexibila la dezvoltare ulterioara
   - UCmd controleaza toate switch-urile elementare

2. Prin realizarea unor subretele de comutare organizate ierarhic, pe mai multe niveluri
   - Reduce numarul de comutatoare(k ≈ log2n comutatoare elementare)
   - Se reduce complexitatea (nC) & creste timpul de comutare (Δt)
   - Comutatoarele elementare sunt de tip Cross Bar (axb), cu a/b mult mai mici decat n/m
   - Intre nivele se aseaza permutarile de interconectare

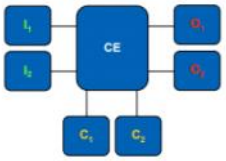


Comutatoare elementare

→ Un comutator elementar are 2 intrari, 2 iesiri si 2 biti de comanda

→ Aceste module se pot integra pe baza permutarilor elementare

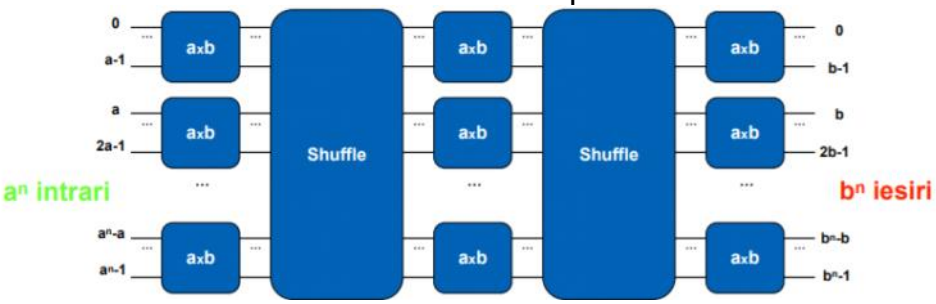
C_1	C_2	Connection	
0	0	$O_1 = I_1; O_2 = I_2$	Direct
0	1	$O_1 = I_2; O_2 = I_1$	Inversat
1	0	$O_1 = I_1; O_2 = I_1$	Broadcast Superior
1	1	$O_1 = I_2; O_2 = I_2$	Broadcast Inferior



Retele de Comutare Ierarhice

1. CrossBar-uri de dimensiuni mici asezate pe mai multe nivele de interconectare
   - Creste timpul de intarziere datorita utilizarii mai multor module – dar se mentine orice intrare - orice iesire
   - Exemplu: structura Beizer(Benes)

2. De tip Delta
   - Sunt comutatoare ierarhice cu a^n intrari si b^n iesiri ce utilizeaza CrossBar-uri (CB de dimensiuni axb)
   - Intre CB-uri se folosesc retele de permutare cu intercalare perfecta (Shuffle)
   - Reteaua Delta are n nivele ierarhice:
     - Nivelul 0 contine a^{n-1} CB-uri de tip axb, pentru a conecta a^n intrari cu a^{n-1}xb iesiri
     - Nivelul 1 contine asadar a^{n-1}xb intrari conducand la a^{n-2}xb module CB
     - Nivelul i contine $a^{n-i}b^{n-i}$ module CB de tip axb



   - Avantajul este ca avem peste tot axb si intre ele nu se schimba RC (acelasi Shuffle)
   - Destinatia este D = (dn-1 dn-2 … d1 d0) cu $0 \leq di < b$
   - Cifrele de reprezentare in baza b, a adresei destinatiei vor controla modulul CB de pe nivelul respectiv (di controleaza CB-ul i)
   - Functia Shuffle-ului este de a interconecta nivelele retelei Delta

Retele Bazate de Rutare

→ Legaturi directe intre o resursa si cele considerate vecine

→ Resursele pot fi distincte dar toate au in comun un router ce se ocupa de comunicarea prin mesaje

→ Comunicarea se face prin mesaje, nu prin adresare directa

→ Nu foloseste comutatea de circuite (CB), ci rutarea

→ Sunt reprezentate prin grafuri
   - gradul nodului - cate canale de conexiune are un nod

- diametru - distanta maxima intre 2 noduri
- regularitatea - nodurile au/nu au acelasi grad
- simetria

→ Caracteristici:
- Topologia - cum sunt legate nodurile
- Rutarea - stabileste calea pe care o ia un mesaj
- Comutarea - mecanismul ce determina cum si cand un canal de intrare este conectat la unul de iesire

1. Topologii Ortogonale

→ Nodurile sunt aranjate intr-un spatiu n-dimensional ortogonal
→ Fiecare legatura produce o deplasare intr-o singura dimensiune

2. Topologii Strict Ortogonale

→ in zona legaturilor P-P
→ Fiecare nod are cel putin o legatura in fiecare dimensiune (pentru a nu face rutare printre dimensiuni)
→ Trecerea intr-o noua dimensiune se poate face din orice nod
→ Nodurile pot fi numerotate folosind coordonatele lor si distanta se poate calcula folosind coordonatele
→ Avantaj: Rutarea e mai simpla si implementarea HW e mai eficienta

Hipercub
- 2 cuburi unul intr-altul
- Din orice nod al cubului interior se poate ajunge pe nivelul superior
- Se poate optimiza drumul

Tor
- Sub forma unui inel, ca un hipercub la care leg calculele
- Torul este mapat pe un hipercub

Retele de tip Arbore

→ Sunt un alt tip de retele de comutare
→ Nu neaparat arbori binari
→ Similar – la structurile cu hipercub, fiecare nod poate fi inlocuit cu un nou cub!
→ Exista mai multe cai posibile pentru mesaje – totdeauna se cauta drumul de lungime minima

Analiza Comparativa Delta /CB

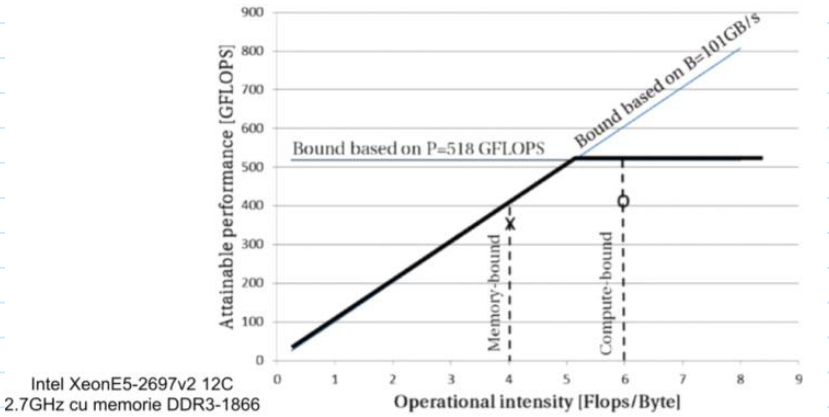
→ Probabilitatea ca o cerere de la o intrare la o iesire sa fie acceptata depinde de m
→ La m mare, scade probabilitatea din cauza confictelor si rejectiilor
→ Cross-Bar da o probabilitate mai mare decat Delta

GPU

Tuesday, June 8, 2021 3:27 PM

Modelul Roofline

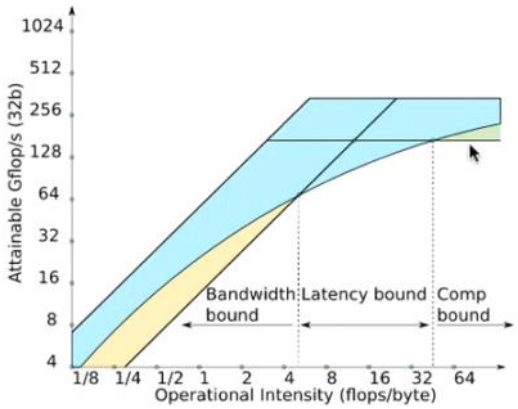
→ Ne spune cum am putea clasifica aplicatiile noastre
→ Vrem sa facem un plot pe care sa punem throughput-ul maxim ca o functie de intensitate aritmetica
→ Intensitatea aritmetica este calculat ca: operatii in virgula mobila / bytes read



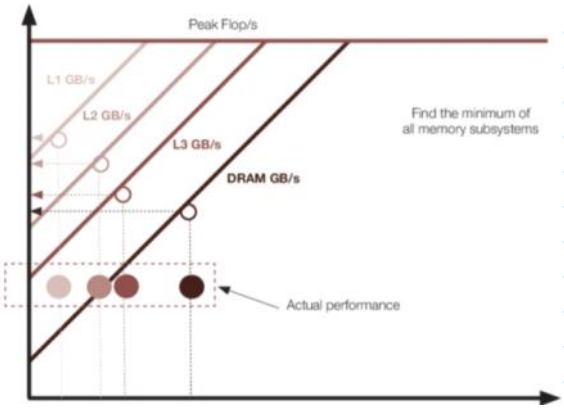
→ Putem calcula limita maxima a unui procesor, adica ce poate sa dea UAL-ul (masurata in GFLOPS)
→ Putem calcula panta graficului care e data de bandwidth si te duce la un throughput al memoriei, deci o limita si aici
→ Pe axa Ox se citeste Operational Intensity care spune:
- val < 5 => esti limitat de memorie (memory-bound)
- val > 5 => incepi sa fi limitat de procesor (CPU-bound)

→ Majoritatea codurilor sunt memory-bound, deci investitia in procesoare cu frecventa mai mare este proasta. Ar trebui sa scriem coduri care sa fie cat mai in zona de CPU-bound
→ Se poate observa ca GPU-urile:
- sunt mult mai performante (ai GPLOPS mai multi)
- au FMA (Fused Multiply Add) care fac adunari si inmultiri in acelasi timp
- au SFU (Special Function Unit) care sunt niste coprocesoare matematice dedicate rezolvarii functiilor complexe

→ Optimizare prin asezarea datelor duce la o utilizare mai buna a memoriei
→ Pt orice procesor se poate intocmi graficul acesta:



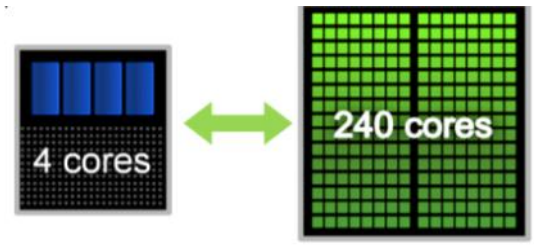
→ Panta depinde de memoria in care poti sa-ti pui datele (Cache L1, L2, LLC sau principala)



Arhitectura GPU

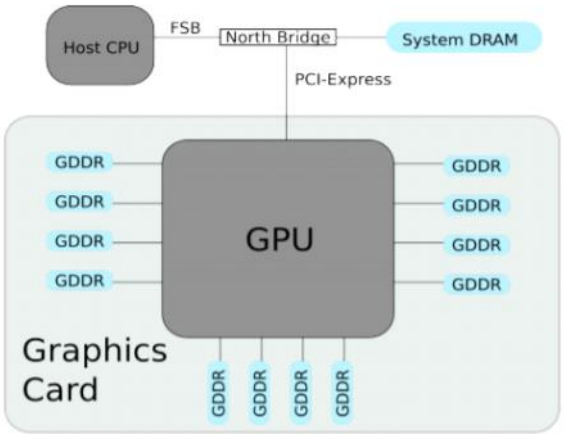
→ De obicei CPU si GPU coexista in sisteme eterogene
→ Accerelatoare matematice
→ CPU
- se ocupa de calculul de uz general
- are UAL-uri solide, coprocesoare solide
- foloseste paralelism cu granularitate mare (coarse-grained) -task-uri, procese
- core-uri putine, mari si puternice

→ GPU
- se ocupa de partea computational intensiva
- au arhitectura orientat catre UAL
- foloseste fine-grained paralellism
- core-uri multe, mai putin inteligente, mai multe operatii executate in acelasi timp



Arhitectura NVIDIA Kepler K40

→ Functioneaza intr-o conlucrare cu CPU



→ Memoria GPU (12GB) este grupata in GDDR (Graphical DDR)
→ GPU-ul este legat printr-un PCI-Express de North Bridge(chipset-ul principal al placii de baza)
→ Prin North Bridge ne legam fie direct la memoria principala, fie la procesor

The Brick Wall

→ Intr-un core, mai putin de 10% din aria cipului e folosit pentru executie reala

→ Densitatea computationala nu mai poate fi crescuta, depinde de consumul de energie

1. Power Wall: procesorul ma costa multa energie
2. Memory Wall: memoria e inceata si consuma mult
3. ILP Wall: nu mai dau randament (Instr Level Parallelism)

Brick Wall = Power Wall + Memory Wall + ILP Wall

→ Solutia consta in introducerea arhitecturilor GPU

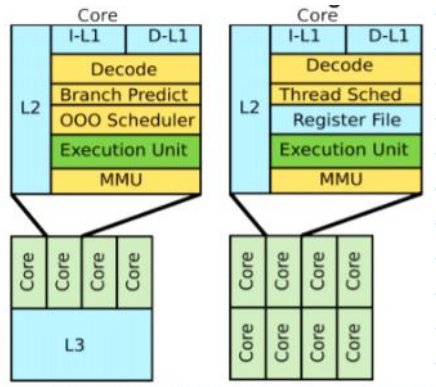
Schimbari de la CPU la GPU

1. Crestere latenta, crestere throuhput

→ S-a scos tot L3 si punem registrii mai multi
→ Astfel a crescut latenta (cam 1%) dar s-a dublat throughput-ul
→ Scotand L3 este o distanta mult mai mare intre memorii off-chip si procesor
→ Ascundem latenta memoriei prin introducerea Fine-Grained Interleaved Threading

Pro:
- cache mai putin
- fara Branch-Predictor
- fara OOO Scheduler (face preemtare care costa mult)

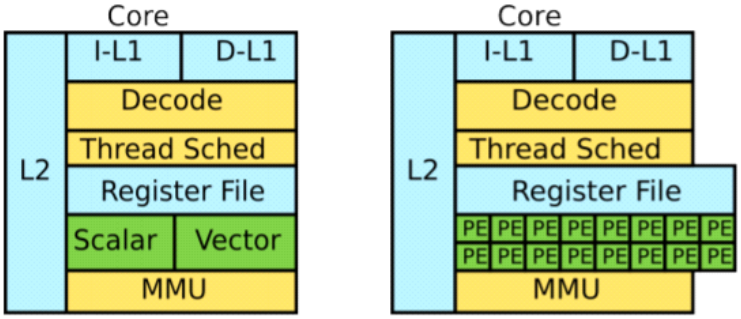
Cons:
- presiune pe registrii
- trebuie sa implementezi Thread Scheduler
- trebuie paralelism la nivel inalt



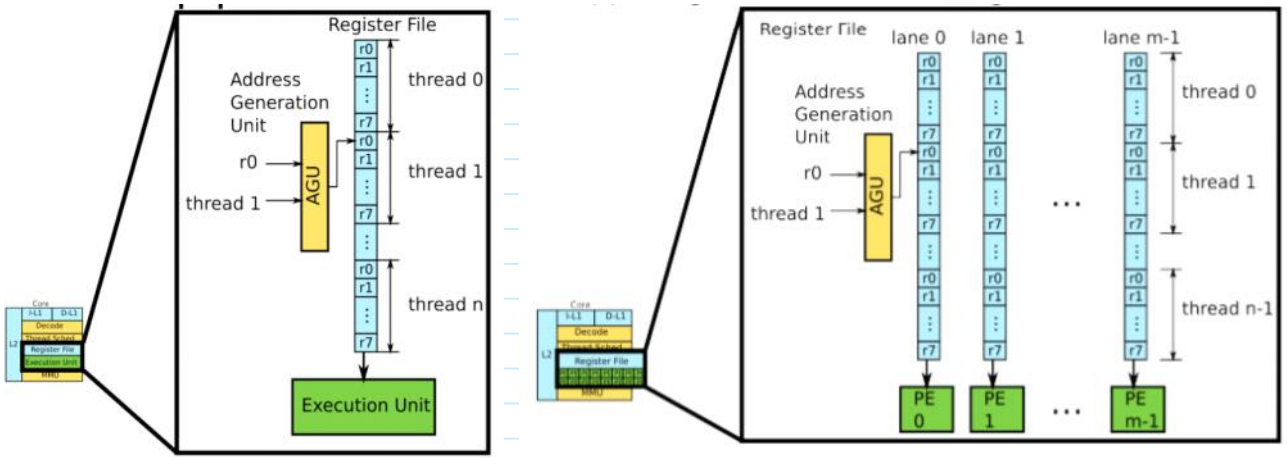
→ Am inlocuit Branch-Predictor cu un Thread Scheduler
→ Am inlocuit OOO Scheduler cu Register Files
→ Registrii vor fi gestionati de niste threaduri si fiind mai multi au loc mai multe date in ei si nu mai trebuie sa ne ducem pana in memoria principala
→ Zero overhead context-switch
→ Threadurile care au date se executa, deci e mai rapid

2. Utilizare SIMD

→ Facem diferenta intre operatii scalare si vectoriale
→ Pe CPU-uri sunt SIMD-uri scurte (de 4)
→ Pe GPU-uri sunt SIMD mai lungi (peste 8)
→ In loc sa am zona de scalar si vector am mai multe PE (Processing Element)



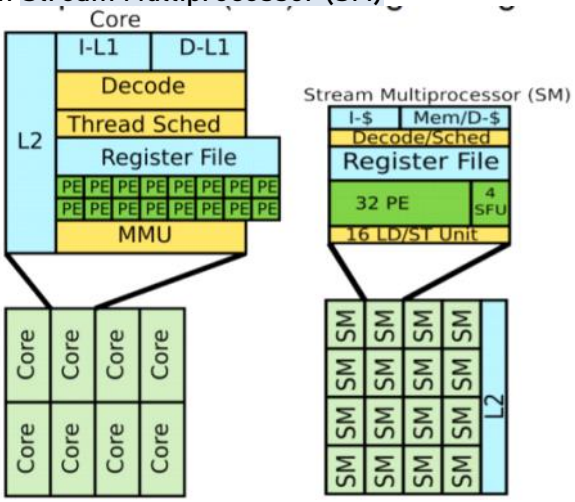
→ Deci avem zero overhead context-switch si multe unitati de procesare
→ Nu ma costa sa schimb intre thread-urile de pe acelasi PE, doar de la threadurile care au date in alti registrii se vor executa pe PE-uri (interleaved threading)



→ La SIMD normal trebuie sa astept dupa date daca nu sunt disponibile, insa aici prin Register Files mult mai multe threaduri au date direct in registrii si se executa imediat
→ Se exploateaza SIMT (Multiple Thread), adica in loc sa fac switch execut threaduri
→ Sincronizarea va fi intre threadurile care au date
→ Adancimea vectorului de procesat este ascuns in spatele multi-threading-ului si comutatea intre ele se face pe baza disponibilitatii datelor in registrii

3. Core-uri Simple

→ Grupam PE-urile intr-o structura mai usoara
→ Se grupeaza intr-un Stream Multiprocessor (SM)



→ L2 cache va fi al intregului sistem, nu local
→ Instruction/Data cache va fi local
→ Va fi nevoie de Decoder si Scheduler (Dar nu un Scheduler prea complicat pentru ca se gestionaza ca un grup de 32PE-uri la un loc)
→ Exista entitatea de FMA (Fussed Multiply Add)
→ Exista SFU (Special Function Unit) care e specializata pe calcule complexe, iar operatiile simple se fac in PE
→ 4 SFU la 32 PE
→ MMU (Memory Control) se face intr-un Load/Store Unit care doar aduce si trimite date

Beneficii GPU

1. Fine-grained interleaved threading (~2x comp density)
2. SIMD/SIMT (>10x comp density)
3. Simple core (~2x comp density)

→ 40x comp density

Programare in GPU

→ Multi-Threading
- Numar masiv de threaduri light-weight(>10000) pe gridul computational
- Ceea ce se introduce in GPU este ideea de multithreading activ astfel incat sa faca contex-switch intre threaduri in functie de disponibilitatea datelor
- Ideea de masca de la SIMD se transfera aici la nivel de thread

1. Vector Programming
   - Unitate programabila vectoriala
   - Inseamna ca pot sa rulez cu w deodata (w este latimea) si se scapa de un for
   - Adica incarcarea datelor in memorie se face linie cu linie
2. GPU Programming
   - Se face o alternanta intre DLR si TLP (paralelism la nivel de date si de thread)
   - Un cod de GPU se numeste kernel computational
   - Se cauta cod complicat si se duce pe GPU si se pune intr-un kernel
     kernelF<<<(4,1), (8,1)>>>(A)
   - Inseamna ca am 4 blocuri si pe fiecare bloc sunt 8 threaduri aman2 de adancime 1
   - Matricea este mapata peste blocuri si mai apoi peste threaduri si apoi urmeaza ca threadurile sa ruleze toate deodata
   - Maparea se face peste SM (se ocupa Schedulerul GPU)

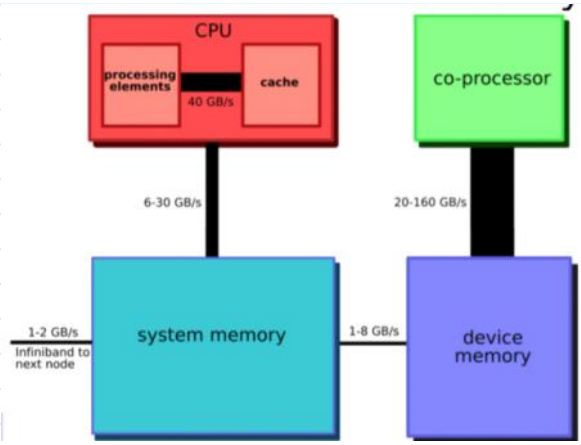
Vector vs. Thread

→ Pro Thread:
- Adancimea modulului de procesare a masinii e transparent lui Instruction Set
- Programatorul nu trebuie sa faca aceasta aliniere (ca la vector)

→ Consta Thread
- Poate fi ineficient sa programam threaduri
- Sincronizare de mana
- E foarte greu de facut debug

Thread Block
- Este un vector de threaduri care execut acelasi program
- Pot comunica pentru a obtine un rezultat
- Are maxim 1024 threaduri si pot avea mai multe dimensiuni (1d, 2d, 3d)
- Un ID de thread are 3 indici
- Fiecare SM poate executa 16 blocuri concurente

Pasi Programare pe GPU
1. Se aloca datele pe CPU si pe GPU, doar spatiu
2. Copiez datele de la CPU la GPU(din System Memory pe Device Memory aici este bottle neck-ul ca el viteza mica)
3. Main step: co-processor se ruleaza treaba (un bandwidth mare deobicei)
4. Se copiaza datele/rezultatele de pe gpu in cpu
5. Trebuiesc puse elemente de sincronizare
6. Dezalocare date



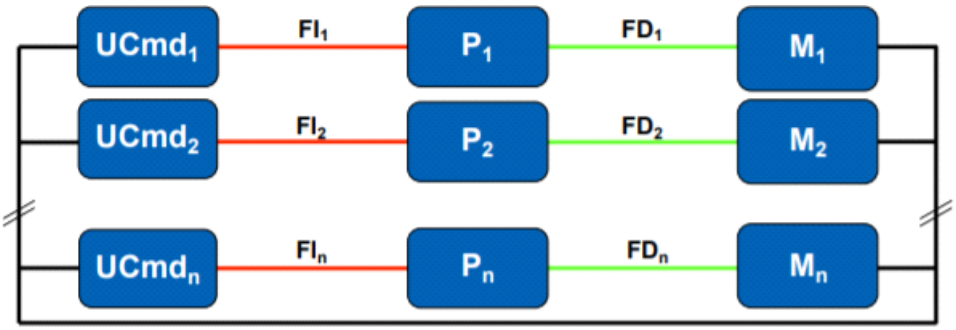
Ierarhie de memorii pe GPU

- Global Memory/ Device Memory: accesibila de toate threadurile cat si de CPU (host)
- Disponiibilitatea datelor este in RAM
- Datele trebuie sa treaca prin VRAM, similar ca la cache
- Fiecare thread are un Local Memory in acei registri, consistenta datelor e pe perioada in care un thread ruleaza
- Host-Memory este RAMul principal
- In Constant Memory se face doar read ca sa fie mai rapid dar se pot pune acolo doar date care sunt de citit
- Shared Memory este memoria blocului si e consistenta cat timp traieste blocul
- Sunt mai mute zona de memorie si e o alta abordare decat la cacheuri

Arhitecturi MIMD

Wednesday, June 9, 2021 2:24 PM

→ E cea mai generala structura pe care o avem (n FI si N FD)
→ Sunt cele mai performante si scumpe
→ Sunt utilizate mai des pentru ca sunt mai flexibile decat SIMD (care e mai tricky)
→ Pot comunica P-P sau P-M, exista cate o unitate de comanda pentru fiecare unitate de procesare



→ Mod de programare:
  - Shared memory (strans cuplate) – Memorie Partajata (e.g. OpenMP, pthreads)
  - Distributed memory (slab cuplate) – Transfer de Mesaje (e.g. MPI)
→ Caracteristici:
  - Structura & arhitectura procesoarelor e eterogena
  - Procesoarele pot functiona autonom ca un sistem universa
  - Mecanisme speciale pentru memorie pentru ca capacitatea de acces a fiecarui P e mai mica decat dimensiunea memoriei
  - Trebuie asigurata protectia memoriei pentru asigurarea integritatii datelor din sistem
→ Modularitate:
  - Expanbilitate (se poate schimba nr de procesoare)
  - Reconfigurabilitate
    - Structurala - distribuirea structurala a functiilor, preluarea sarcinilor de alte procesoare
    - Arhitecturala - schimbarea caracteristicilor de baza (UAL, registrii)
→ Probleme:
  - Constructia algoritmilor: manual sau automant prin limbaje de progr. concurenta
  - Planificarea pentru executie a taskurilor
  - Sincronizarea taskurilor
  - Siguranta in functionare prin disponibilitate si integritate

Sisteme MULTIPROCESOR

→ Sistem de calcul unitar unde o colectie de procesoare autonome functioneaza concurent si coopereaza la realizarea unei functii logice si de calcul de sub controlul unui "executiv" unic
→ Au un singur spatiu de adresare - comunicare prin Shared Memory

1. Cu memorie distribuita / scalabilitate:
   - Gestiune dinamica a accesului la memoria distribuita
   - Gestiune statica a accesului la memoria distribuita
2. Cu memorie centrala / nu exista scalabilitate

- Comunicarea se face prin RC (e.g. crossbar switch)
- Conexiuni de tip magistrala (bus) multiplu

Sisteme MULTICALCULATOR

→ Mai multe calculatoare autonome, interconectate, ce colaboreaza la realizarea unor task-uri din cadrul unei lucrari complexe

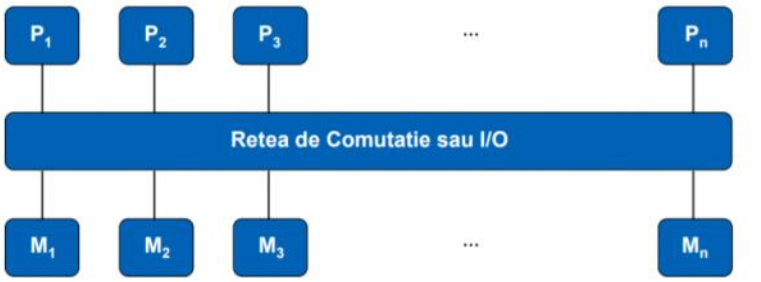
→ Au spatii multiple de adresare - comunicare prin transfer de mesaje

1. Multicalculatoare Distribuite legate prin
   - Mesh – plasa de interconectare P-M, NEC…
   - Conexiuni Butterfly (Perfect Shuffle) – CM5
   - Hypercube – la sistemele NCUBE, BlueGene…
   - Retele LAN rapide: Myrinet, ATM, GigaEthernet, InfiniBand, MPLS …
2. Multicalculator Centralizat - tipul Cluster

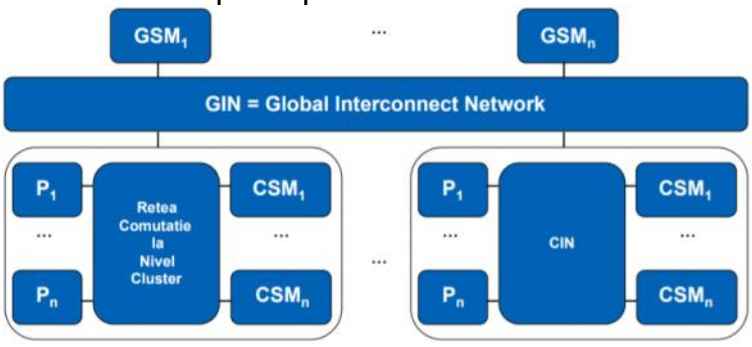
Sisteme cu SHARED MEMORY

→ Implementate in una din urmatoarele configuratii: UMA, NUMA, COMA

1. UMA - Uniform Memory Access
   - Fiecare nod din sistemul are acces la oricare date, iar coerenta este vazuta de toti ceilalti
   - Nu exista o incoerenta a datelor -> usurinta in programare si viteza
   - Accesul este unic: orice procesor se poate conecta la un modul de memorie in mod uniform
   - Reteaua de comunicatie (I/O) poate fi de orice tip: CB sau Delta



2. NUMA - Non-Uniform Memory Access
   - Distributia datelor e facuta in asa fel incat nu se garanteaza consistenta datelor
   - Trebuie mecanisme explicite pentru a face read

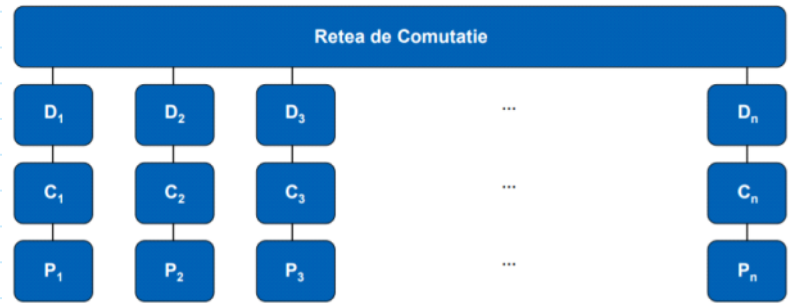


- CSM = Cluster Shared Memory
- GIN = Cluster Interconnect Network
- GSM = Global Shared Memory
- GIN = retea de comutare globala la care se conecteaza clustere formate din procesoare

- Procesoarele sunt grupate in clustere care au fiecare GSM conectat prin RC la nivel cluster
- Pentru a comunica intre clustere exista un GSM care se conecteaza prin GIN
- Accesul la memorii depinde de modulul unde trebuie efectuat accesul (CSMi sau GSMj)

3. CC-NUMA - Cache Coherent NUMA
   - Se asigura coerenta doar la nivelurile de cache, nu si la memoria principala
   - Coerenta accesului la memorie e mai bine stabilita ca la NUMA
   - Se ruleaza mai rapid cand se ruleaza in cadrul intern (pentru datele aduse in memoria ta locala) si nu mai trebuie sincronizare la nivelul intregului sistem

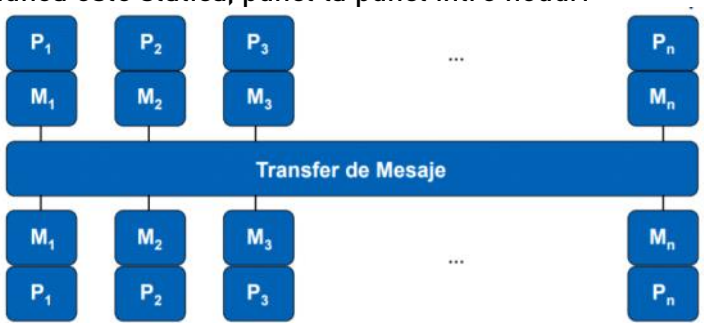
4. COMA - Cache-Only Memory Architecture
   - Exista doar cache-uri
   - Procesoarele nu au acces direct
     - la propria zona de date
     - la cea a celorlate procesoare
   - Procesoarele au acces direct doar la Cache



   - Datele se transfera prin RC
   - Daca vrem date de la alt procesor, acestea trebuie aduse din cache-ul procesorului in cache-ul local (D2 -> RC -> D1 -> C1)
   - SW COMA - Software Cache Only Memory

Sisteme cu DISTRIBUTED MEMORY

→ Accesul se face prin transfer de mesaje
→ Nodurile sunt legate printr-o retea de comutatie
→ Procesoarele au acces doar la memoria locala
→ Conexiunea este statica, punct la punct intre noduri



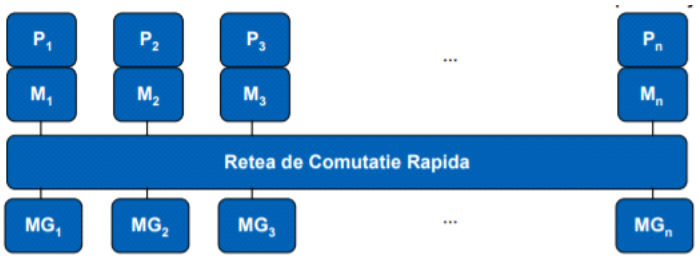
Sisteme STRANS Cuplate

→ Dispun de procesoare Pj cu memorie locala asociata Mi de tip multiport
→ O retea de comutatie rapida RC (dar fixa) ce permite accesul uniform la memoria globala partajata MGi
→ Comunicarea si sincronizarea se face prin MGi cu o rata de transfer similara cu a

memoriei locale

→ Asigura o scalabilitate redusa datorita faptului ca RC este fixa si de aceea au o structura dedicata

→ Performantele sistemului sunt date de modul de acces la memoria partajata



Sisteme SLAB Cuplate

→ De tip cluster → calculatoarele au resurse locale necesare functionarii independente

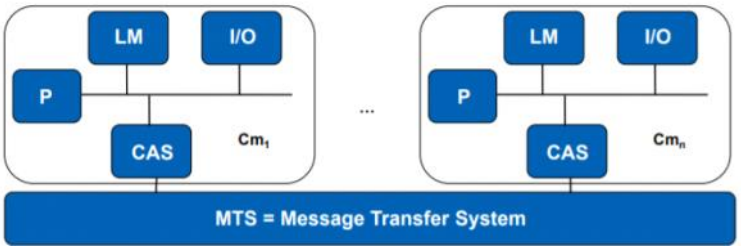
→ Procesele se executa pe procesoare care dispun de toate resursele necesare

→ Comunicare prin

- Transfer de mesaje
- Comutare de circuite

→ Sunt eficiente cand exista interactiuni minime intre sisteme (Embarassingly Parallel)

→ Multicalculatoare centralizate



- CMi = Computer Module
- LM = local memory (acces din interior/exterior)
- I/O = magistrala de intrare/iesire (acces din interior/exterior)
- MTS = Message Transfer System
- CAS = Collision Avoidance System

→ MTS poate fi implementata cu comutare de circuite sau transfer de mesaje

→ MTS dicteaza performantele prin lungimea mesajelor & viteza de lucru

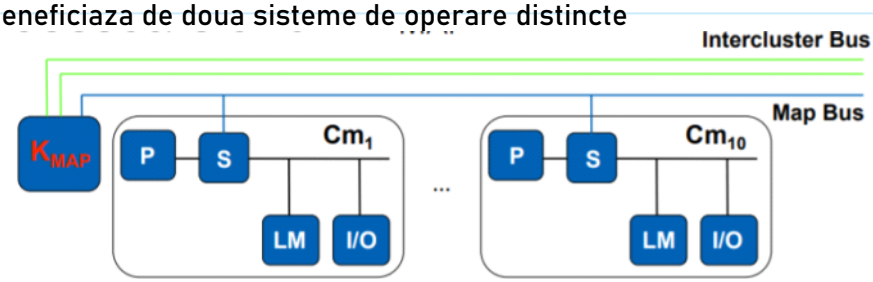
→ CAS - legatura si accesul la MTS se fece prin excludere mutuala

- Un singur transfer odata daca MTS este magistrala
- Mai multe transferuri simultane daca MTS este de tip retea

Arhitectura CM*

→ Cm* e proiectat cu intentia de a obtine in acelasi timp performante bune si fiabilitate sporita

→ Cm* beneficiaza de doua sisteme de operare distincte

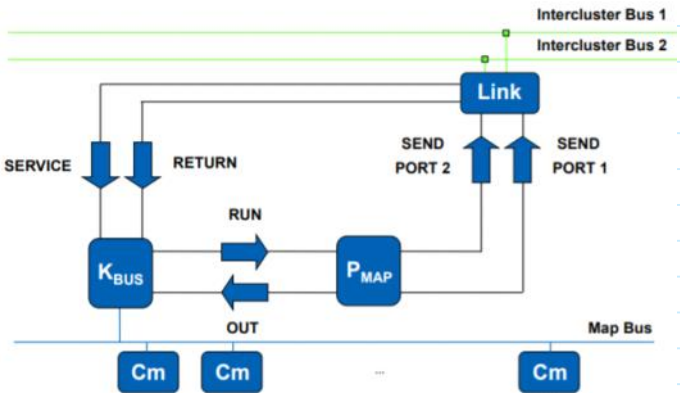


→ Conecteaza mai multe Computer Module (CM) individuala printr-o magistrala

→ Structura multipla de calculatoare -> cluster
→ Un cluster e format din mai multe CM, KMAP si Map Bus
→ Toate cererile pt CM-uri sunt facute pe Map Bus si sunt gestionate de KMAP
→ Poate sa execute task-uri independente in paralel
→ Mai exista si 2 Intercluster Bus prin care se poate comunica cu alte clustere intr-o Structura Matriceala de Clustere

1. Switch S
   - Trebuie sa primeasca cererile de la P si decide daca are nevoie la memoria locala sau la I/O
   - Da legatura la alte resurse CM
   - Se ocupa de translatie a adreselor la nivel local
   - Formateaza adresa externe prin concatenarea adresei in cadrul paginii si numarul de identificare al procesorului
   - Memoria este paginata

2. KMAP
   - Asigura controlul in cadrul unui cluster
   - Asigura comunicarea intre clustere prin Intercluser Bus
   - Functiile KMAP:
     - Maparea adreselor (adresele locale -> Map Bus, adresa globala -> Intercluster Bus)
     - Gestionarea circuitelor (pt a face acces pe Map Bus)
     - Transmiterea mesajelor pe Ethernet , TokenBus etc
     - Sincronizarea



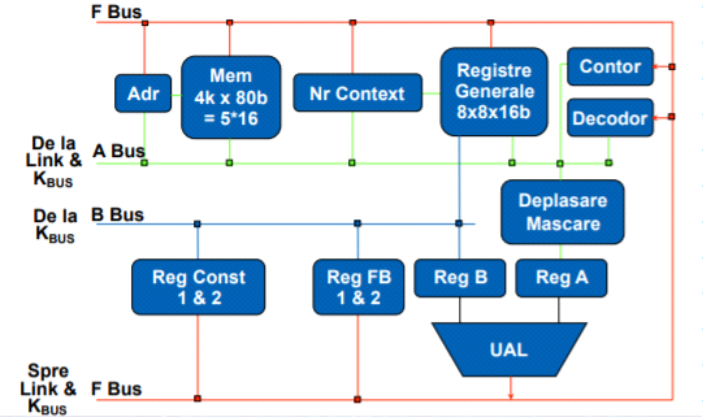
   - Un CM genereaza o cerere pe care il pune pe Map Bus
   - KBUS care primeste si gestioneaza aceasta cerere
   - Exista procesorul PMAP care ia decizia daca sa execute cererea (prin cozile Run si Out se face decizia)
   - Daca cererea CM-ului e catre un alt cluster, se trimite catre exterior
     (CM -> Map Bus -> KBUS -> PMAP -> SEND 1/2 -> LINK -> Intercluster Bus)
   - Exista 2 tipuri de conexiune:
     - De circuite – in cadrul cluster-ului
     - De mesaje – intre clustere

3. Comunicatie KBUS - PMAP
   - KBUS poate mentine starea a 8 contexte ce manipuleaza cereri, le aloca cereri si sunt plasate in coada RUN
   - PMAP este cel care le preia, prelucreaza cererea si translateaza adresa daca e cazul
   - In acest timp KBUS poate citi alte cereri si efectua alte functii cerute de PMAP
   - In PMAP se executa schimbarea de context din RUN in OUT
   - Tot KBUS ia ce este in OUT si transfera la adresa destinatiei si confirma procesul

4. Comunicatie LINK - PMAP
   - Daca CM pune o cerere catre exterior, KBUS transmite catre PMAP
   - PMAP vede ca adresa e la exterior si trimite la LINK si suspenda contextul
   - Raspunsul se intoarce in RETURN care se duce in KBUS ce il trece in RUN
   - Ajunge in PMAP si el stie ca trebuie sa il trimita CM-ului care a facut cererea
   - KBUS primeste o noua cerere de alocare a unui nou context in RUN

5. Comunicatia KBUS - MAP BUS
   - KBUS trateaza si erorile
   - Map Bus este o magistrala de comunicatii
   - KBUS controleaza toate accesele pe Map Bus, precum si cozile si registrele pentru comunicarea PMAP

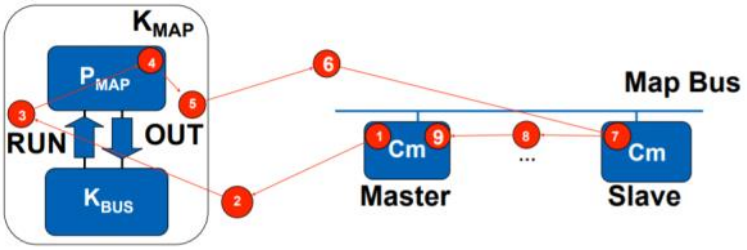
6. PMAP
   - Procesor dedicat, microprogramat
   - Functii:
     - Translatarea adreselor nelocale din cluster
     - Gestioneze niste primitive SO ca sa faca transferul
     - Culegerea informatiilor statistice (ca la PC)



   - Are mai multe magistrale pentru legaturile dintre PMAP si cozi (Link, KBUS)
   - Are niste registrii si buffer care fac legatura cu UAL
   - Deplasarea si Mascarea sunt pentru a putea face schimbarea de context

Caracteristici Cluster:
- → Faciliteaza cooperarea intre resursele sist
- → Bottleneck-ul este Map Bus-ul (o operatie pe rand)
- → Orice referire la memorie este tratata de KMAP (rutare)
- → Clusterele comunica intre ele prin KMAP conectate la Intercluster Bus

Etapele de acces in cadrul unui Cluster



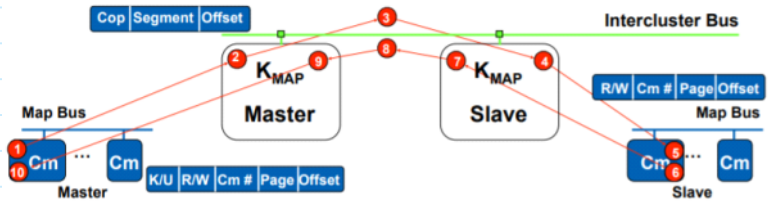
1. In CM Master se initiaza acces la o memorie nelocala
2. CM Master emite adresa virtuala catre KBUS

3. KBUS activeaza un context (crearea structurii de date specifica tranzitiei) ce se pune in coada RUN catre PMAP
4. PMAP trateaza contextul si face translatarea adresei
5. PMAP pune in coada OUT o cerere pentru ciclu de memorie pentru CM Slave din cluster-ul curent
6. KBUS trimite adresa fizica catre CM Slave prin Map Bus
7. S-ul local din Cm Slave face acces la memoria locala prin furt de ciclu
8. KBUS "permite" ca rezultatul operatiei de acces la memorie sa fie furnizat catre CM
9. CM Master preia datele, termina operatia in curs si continua executia

Formatul mesajelor

→ U/K – User/Kernel
→ R/W – Read/Write
→ Cm # – numarul Cm-ului ce efectueaza transferul
→ Page – utilizata pentru translatarea locala
→ Offset – In cadrul paginii, adresa relativa

Comunicare Inter Cluster



1. CM Master trimite o cerere de transfer la KMAP Master
2. KMAP Master pregateste mesaj/pachet intercluster si codifica cererea
3. Mesajul intercluster e transmis pe Intercluster Bus pe baza algoritmilor de rutare
4. KMAP Slave decodifica cererea primita si o trimite catre Clusterul sau local
5. Cererea de ciclu de memorie este trimisa la CM Slave
6. CM Slave transmite rezultatul catre KMAP Slave
7. KMAP Slave pregateste mesajul intercluster - reactivarea contextului respectiv
8. KMAP Slave transmite rezultatul catre KMAP Master
9. KMAP Master receptioneaza si interpreteaza mesajul primit
10. Rezultatul este trimis catre CM Master

Debugging si Profiling

Tuesday, June 8, 2021 8:27 PM

Debugging

→ In primul rand codul trebuie sa functioneze bine si sa dea rezultate
→ Apoi se discuta de eficienta lui

1. Analiza
2. Design (Introducere Threaduri)
3. Debug pentru corectitudine
4. Analiza de performanta (si sa ramana corect)

Dezvoltare de cod paralel

→ Paralelizam doar zonele care merita paralelizate
→ Paralelizare prin: Threads, CUDA, OpenMP, Cilk, TBB
→ Se verifica ca merge corect si dupa se optimizeaza

Advisor Workflow

→ 5 pasi de a transforma cod serial in paralel
→ Multi algoritmi nu sunt usor paralelizabili
→ Poti intotdeauna sa modifici schimbarile

Analiza Hotspot

→ Identific o bucata de cod unde programul isi petrece timpul
→ Se foloseste Parallel Amplifier sa gasim acele hotspot-uri
→ Se cauta bucle care se pot paraleliza
→ Programul arata cat de mult foloseste codul core-urile
→ Corectitudinea se poate verifica cu Parallel Inspector care arata race conditions si deadlocks
→ Cum e facut un cod? Un cod are o zona clasica de date si de control/instructiuni
→ Data raceurile sunt un fel de control al datelor si deadlockurile se ocupa mai mult de threaduri/instructiuni

Debugging Tools:

→ Plug-in to Microsoft* Visual Studio*
→ OpenMP*, Intel® Threading Building Blocks, Win32* threaded software
→ Izoleaza problemele, nu simptomele

Parallel Inspector

→ Analiza se face cu monitorizarea:
  - threadurilor si API-ului
  - ordinea de executie a threadurilor
  - accesul de memorie intre threaduri
→ Adauga call-uri catre o biblioteca care inregistreaza informatii
→ Se creste timpul de rulare si marimea ocupata

Binary Instumentation

- Cand linkeditam o biblioteca precompilata (binar fara cod)
- Se urmareste cu niste programe mici prin cod si cand ave m interactune cu codul din biblioteca precompilata intra in vigoare instrumentarea
- Se vede ce fac instructiunile asupra datelor cand se face instrumentarea

Parallel Amplifier

→ Arata cat asteapta threadurile(idle) si unde asteapta mult
→ Arata data races

Performance Issues
→ Load Balance
- Distributie neuniforma de lucru paralel si timp pierdut
- Rezolvare:
  - redistribuirea muncii
  - asignare statica (sa aiba toate threadurile acelasi volum de munca daca sunt la fel, daca nu sa fie programate static)
  - asignare dinamica (rupe task-uri mari in bucati mai mici si le da threadurilor care stau degeaba a.i. toate au ceva de facut)

→ Sincronizare
- Folosirea excesiva a datelor globale
- Folosire de instrumente de sincronizare
- Sectiunile cu Lock fac ca thread-urile sa stea mult in idle
- Se poate rezolva prin eliminarea sincronizarii, sa folosim variabile locala si doar update-ul se face in datele globale
- Se pot folosi functii atmice de cate ori e posibil

→ Parallel Overhead
- Timp necesar de creare si programare threaduri

→ Granularitate
- Nu este sufient de mult cod paralelizat

Profiling si Tracing
→ doua moduri prin care se pot extrage date

1. Profiling

→ Statistici de analiza a performantei, arata performanta bottlenecksurilor si hotspoturilor
→ Se gasesc locurile care au o influenta asupra performantei
- De cate ori a fost apelata ceva
- Timpul inclusiv si exclusiv
- Hardware Performance Counters
- Modul de folosire a memorie, comunicatii

→ Solaris Studio, Total View
→ Sampling
- Intreruperi periodice ale SO si se obtin niste samplings care se salveaza cu timestamp in niste directoare care se analizeaza apoi
- Se construieste o histograma cu samples din PC

→ Masurare
- Inseram in cod de masurare inainte si dupa regiunile de interes

2. Tracing

→ Cand si unde au avut loc evenimentele in timeline-ul codului
- Inregistreaza informatiile despre diferite puncte/evenimente pe parcursul executiei
- Timestamp al evenimentelor

→ Se genereaza foarte multe date
→ Fata de profiling exista posibilitatea sa ratezi o info importanta
→ Este pentru stresarea analizei dinamice din coduri

Performance Counters (PC)
→ Zone de registrii si de memorii care salveaza date despre ce se intampla intr-un procesor

→ La inceput se foloseau pentru a vedea corectitudinea hardware
→ Ofera informatii despre:
  - interactiunea cache (cache miss sau page miss la mem principala)
  - branching, pipeline, eficienta floating-point
  - intructiuni per ciclu
→ Exista Api-uri prin care se citesc aceste date: PAPI
→ PAPI (Performance API)
  - ne ofera un mod de accesa PC pe mai multe arhitecturi
  - Evenimentele Countable sunt de 2 tipuri
    - Platform-neutral Preset Events (e.g., PAPI_TOT_INS)
    - Platform-dependent Native Events (e.g., L3_CACHE_MISS)
  - Evenimentele Preset(construite de mana) pot deriva din evenimentele native
    - PAPI_L1_TCM (Total Cache Misses) = L1 Data Misses + L1 Instruction Misses
    - Trebuie avut grija la denumiri
  - Interfete PAPI:
    - Hardware Specific Layer
    - Portable Layer
    - 3rd Party and GUI (API care ofera vizualizarea statisticilor)
  - Tools pentru PAPI:
    - Scalasca, KOJAK (FZ Juelich, UTK)
    - TAU (UO)
    - PerfSuite (NCSA) etc
→ PC sunt disponibili prin PAPI si pe ratele, memorie, switch, network etc

Analiza Performante

Wednesday, June 9, 2021 12:28 PM

→ Analiza Performantei
→ Provocarile Corectitudinii Paralele

Analiza Performantei de Date

→ Se strang informatii date de Performance Counters si se analizeaza de mana
→ Ne dorim automatizarea analizei
  - Pentru ca unele date sunt prea multe (ordin de G/Tbytes)
  - Daca avem multicore-uri
  - Vrei sa mearga tool-ul pe mai multe arhitecturi

→ Pentru automatizare se incearca folosirea unor unelte care sa scoata flag-uri despre ce il intereseaza pe utilizator
→ Tools: Paradyn, KOJAK, Scalasca, Periscope

1. Trace File Visualization

- Analiza in sisteme distribuite (MPI)
- Problema constau in Late Sender/Receiver si Wait at Barrier
- Ne da un raport generic intre cat se fac operatii utile si comunicare (MPI)
- Ne arata exact statisticile despre comunicarea prin mesaje
- Vampir, Jumpshot, Paraver
- Pasi:
  - Afiseaza procesele selectate
  - Afiseaza schimbarile de stare (culoari de activitate)
  - Arata mesajele si Operatii pe MPI-IO
  - Poate arata si paralelismul

2. PerfExplorer

- Tot la MPI
- Rescrie directia unei relati liniare intre 2 variabile(evenimente)
- Utile daca vrem sa vedem cand se intampla corelatiile

3. Scalasca / KOJAK

- Iti arata zonele relevante pentru ce il intereseaza pe utilizator
- Pentru utilizatori normali
- Obtinem o productivitate mai mare pentru procesul de analiza a performnatelor
- Cube
  - Arata un call graf: toate functiile din cod
  - Arata si system tree-ul adica threadurile care au intrat in fiecare functie
  - Si timpul threadurilor (execution, idle...), apoi pot vedea unde se petrece acel timp(pe comunicrea mpi pe threadul 0 de ex)
  - Arata si patternurile de late receiver si late sender
  - Poti vedea si unde asteapta threadurile (idle threads)

Provocarile Corectitudinii Paralele

→ Paralelismul introduce un nou set de probleme: data race, deadlocks si sunt mult mai greu de identificat
→ Unele probleme pot aparea foarte rar, deoarece programul e pe threaduri, la fiecare

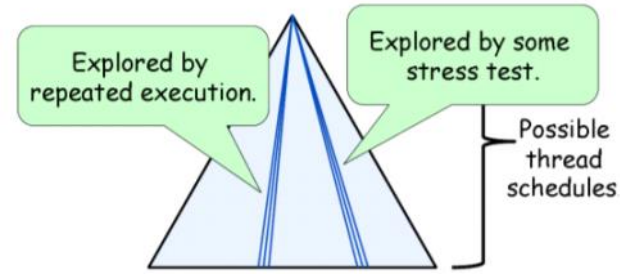
rulare Thread Scheduler este diferit si f rar pot ajunge sa am mereu aceasi configuratie pentru a determina bugul care poate aparea foarte rar

→ Pot rezolva asta setand un load factor, e incarcarea resurselor

→ Cand un SO are un load factor heavy > 80%, se afla intr-o zona de stres mare si atunci pot vedea probleme programului pentru ca se schimba politica de Scheduling

→ Nu e prea mult random in SO, deci unele bugguri pot sa apara doar pe anumite sisteme

1. Analiza Dinamica
   a. Stress Testing prin oversubscription pe masina ca sa obtinem scheduling diferit
   b. Noise Making / Random Scheduling
      - Executa cu politici de Scheduling diferite
      - Poti ajunge la cod suspicios pt ca seamana cu coduri cu virusi/cai troieni
      - Se pot folosi tool-uri care sa controleze Scheduling-ul si exploreaza Scheduling mai variat
      - Limatari:
        - Tot raman multe politici de Scheduling care raman netestate
        - Se poate face combinatie de Stress Testing si Random Scheduling



   c. Race Detector/ Race Pedictor
      - Se face un Trace pe rezultatul rularii codului si ne intrebam daca s-ar putea intampla un Race Condition
      - Se analizeaza locatiile de memorie accesate de threaduri diferite, deci S-AR PUTEA sa avem o coliziune
      - Se streseaza programul pentru a descoperi probleme
      - Tools pentru: data race-uri, atomic violations, deadlocks, consistenta memoriei

2. Analiza Statica

→ Analizam o executie/trace real al unui program

→ Se descopera data races is deadlocks fara sa rulam programul
   - Se analizeaza doar codul sursa
   - O acoperire mai mare
   - Se pot obtine multe false warnings

3. Active Random Testing

→ Combinare analiza statica si dinamica

→ Fac analiza unde AR PUTEA aparea erori si introduc in Random Testing posibilele erori

→ Analiza Predictiva iti poate zice unde sunt bug-urilor si da multe false positives

→ E time-consuming pentru ca e greu sa determini daca chiar sunt false positives

→ Se foloseste analiza predictva sa se gaseasca potentialele bug-uri si dupa se face Biased Random Teasting sa verificam fiecare bug

→ Se da un raport daca se intampla data races, insa daca nu se gaseste ceva gresit nu insemana neaparat ca nu exista

→ Exista mai multe tool-uri: CalFuzzer

BenchMarks

Wednesday, June 9, 2021 6:35 PM

- → Sunt niste teste ce permit compararea performantelor intre sisteme/arhitecturi
- → Aceste teste constau in seturi de programe care sunt menite sa scoata in evidenta performantele sistemului de calcul:
  - bandwidth, latenta
  - CPU performance cu floating point operations
  - Memorii externe
- → Tipuri de benchmark:
  - Aplicatii reale:procesoare de text, aplicatii urilizator
  - Kernel: contine doar parti esentiale dintr-o varietate de programe : Linpack
  - Micro-benchmarkuri: testeaza componente ale sistemului
  - Pentru sisteme paralele Sintetice: se strang date statistice asupra operatiilor efectuate de aplicatii si se construiesc programe
  - Pentru sistem I/o

Latency vs. Throughput

- → Latency (timpul de executie): timpul necesar terminarii unui task anume
- → Throughput / Bandwidth (latimea de banda): numarul de taskuri/transferuri intr-un timp fix
- → La paralelism se exploatea throughput
- → Deseori latenta si throughput sunt contradictorii
- → SpeedUp
  - A este de x ori mai rapid ca B daca
    - x = latency(B) / latency(A)
    - x = throughput(A) / throughput(B)
  - A este cu x% mai rapid ca B daca
    - latency(A) = latency(B) / (1 + x/100)
    - throughput(A) = throughput(B) * (1 + x/100)
  - SpeedUp = P / G

BenchMarks

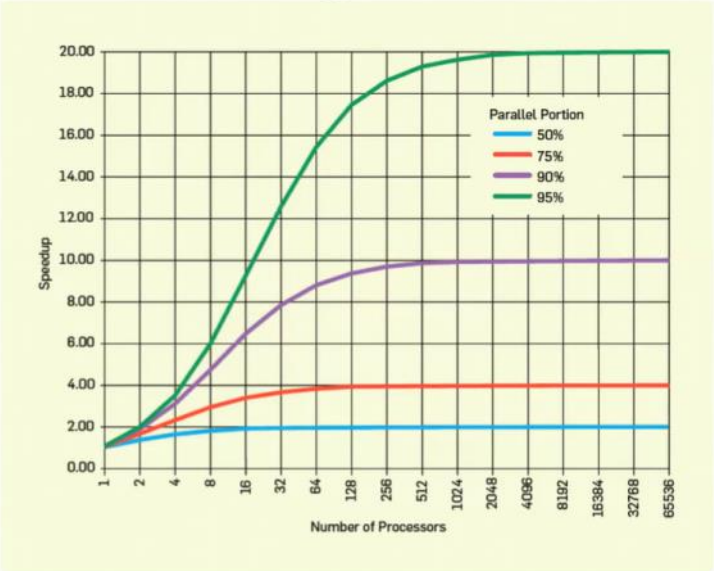
- → Reprezinta un set standard de workload-uri
- → Utilizate pentru a compara performanta mai multor sisteme
- → Reprezentative programelor reale sau chiar programe reale

Micro-Benchmarks

- → Set non-standard de non-workload-uri
- → Programe mici utilizate pentru izolarea unor chestiuni de performanta
- → Nereprezentative pentru manifestari complexe: turnuri Hanoi, 8-queens, etc

Legea lui Andahl

- → Spune ca speedup-ul este limitat de timpul necesar rularii secvential din cod, chiar daca paralelizam



→ Obtinem un speedup de maxim 2 pt 50% paralelizare
→ Obtinem un speedup de maxim 4 pt 75% paralelizare
→ Obtinem un speedup de maxim 10 pt 90% paralelizare
→ Obtinem un speedup de maxim 20 pt 95% paralelizare
→ Daca vrem sa profitam de arhitecturile moderne trebuie sa folosim paralelism la maxim

Benchmark-uri pentru masini seriale

→ SPEC (Standard Performance Evaluation Corporation
  - Realizeaza benchmarkuri pentru siteme de calcul si le publica
  - Testeaza performantele unui server pentru cereri HTTP paralele (SPECweb)
  - Testeaza CPU-ul ruland 29 de aplicatii INT si FP (SPEC CPU)
  - SPEC sunt scrise in C si in FORTRAN
  - SPEC 150 e de 1.5x mai rapid decat SPEC 100
→ GeekBench
  - Pentru sistemele cu mai multe core-uri, dar un singur sistem de calcul
  - Salveaza rezultatele online
  - Problema workload-uri simple ce nu "streseaza" complet sistemele
→ TPC, BAPCo, Sandra
  - TPD - Tranzactii de baze de date
  - BAPCo - Pentru aplicatii pe mobil si testare de sistem intreg
  - Sandra - Sistem Analyser, testare si analiza

Benchmark-uri pentru masini paralele

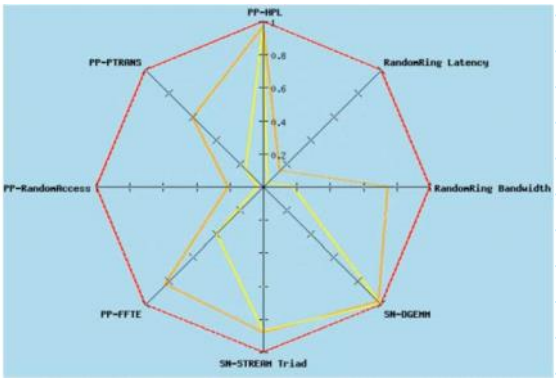
→ LINPACK
  - Extrem de raspandit si sunt disponibile pentru toate sistemele relevante
  - Face operatii cu algebra liniara, este CPU-bound
  - Masoara cat de repede rezolva computerul un sistem dens de ecuatii liniare folosnd eliminare Gaussiana O(n^3) : A·x=b de dimensiune n · n
  - Aceste operatii genereaza foarte multe operatii in virgula mobila
  - Avantaje
    - Se obtine un singur rezultat (optinut si estimat)
    - Usor de definit si usor de clasificat pe baza lui
    - Permite modificarea dimensiunii problemei
    - Disponibil online
  - Dezavantaje
    - Scoate in evidenta dar viteza, nu numarul de procesoare

- Nu evidentiaza bandwidth, RC, mecanisme de tipul gatter/scatter

→ HPC Challenge
- Un benchmark care evidentiaza:
  - procesoarele, memoria, reteua de interconectare
- Avantaj: arhitecturile sunt descrise prin mai mult decat FLOPS-urile de la LINPACK
- Scopuri
  - Complementar cu TOP 500
  - Verifica caracteristicile accesului la memorie
  - Permite optimizari
  - Rezultate arhivate si verificabile
- Tipuri
  - Local: un singur procesor participa la calculele dorite
  - Embarrasingly Parallel (EP): fiecare procesor din sistem participa la calcule, insa ele nu comunica explicit intre ele
  - Global: toate procesoarele din sistem participa la calcule si comunica explicit intre ele
- E format din 7 benchmark-uri relevante
  1. HPL (LINPACK) - MPI Global Ax=b
  2. STREAM - Local, *STREAM - EP A <- B + s*C
  3. PTRANS - MPI Global A <- A + Bt
  4. RandomAcces - Local, MPI Global (*RandomAccess - EP)
  5. Bandwidth si Latency - MPI
  6. FFT - Local, Global, EP Z <- fft(X)
  7. Matrix Multiply - Local, EP
- In HPCC se pot adauga frameworkuri suplimentare daca e nevoie
- HPCS incearca prin HPCC
  - Uniformizeze diferentele existente in ierarhia de memorie
  - Imbunatateasca performantele aplicatiilor reale
  - Usureze programarea sistemelor (paralele) de calcul

→ Altele: SPLASh2, NAS,SPECopenMP

Diagrame Kiviat



→ Performantele sunt total distincte pentru procesoare asemanatoare
- Linpack (rosu) - aproximativ la fel pt toate
- RandomRing (portocaliu) - diferente mari

→ Performantele intre clustere sunt foarte diferite